VOID HACKS() 8.0 · TRACK: ABHEDYA — CYBER SECURITY & DIGITAL FORENSICS

Product Requirements Document

अभेद्य चक्र

Operation “Abhedya-Chakra” — a local-first cyber-financial investigation and digital-forensics platform that turns 2,000,000 banking transactions into an explainable money trail, a layered mule network, and court-ready investigative documentation.

VERSION	2.1 — dataset-grounded
STATUS	Approved for implementation
THEME	Cyber Security & Digital Forensics
ASSOCIATED ORG	Indore Police Commissionerate
EXECUTION WINDOW	36 hours
DATASET	VoidHacks8_MuleAccount_2M_Transactions.csv · 2,000,000 rows · ≈24,873 accounts · 2026-09-15 → 2026-09-29
SUPERSEDES	v2.0 — see §32 changelog, §33 consequences
TEAM	4 members · Technical Lead owner
DATE	01 October 2026

[PS] problem statement · [DEC] team decision · [ASSUME] assumption · [DATA] dataset-derived

Void Hacks() 8.0 | Track: Abhedya — Cyber Security & Digital Forensics

Field	Value
Document version	**2.1 (dataset-grounded)** — supersedes 2.0 (build-ready)
Status	Approved for implementation, pending mentor answers in §29 and profiler verification in §7
Event	Void Hacks() 8.0
Theme	Cyber Security & Digital Forensics
Associated org	Indore Police Commissionerate
Execution window	36 hours
Dataset	`VoidHacks8_MuleAccount_2M_Transactions.csv` — 2,000,000 rows, 11 columns, ≈274 MB, ≈15-day window
Codename	`abhedya-chakra` (अभेद्य चक्र — "the impenetrable wheel")
Team size	4
Owner	Member 1 (Technical Lead)
Changelog	§32 · **Dataset consequences** §33

> **How to read this document. [PS]** — derived from the official problem statement; non-negotiable. **[DEC]** — our implementation decision; locked for this build, changeable by the Technical Lead. **[ASSUME]** — a working assumption standing in for an unanswered question (§29); each has a declared fallback so no open question blocks the build. **[DATA]** — new in v2.1. An observation about the supplied CSV. **Every [DATA] figure in this document is PRELIMINARY until `make profile` reproduces it** (§7.1, FR-A05). Preliminary figures must never be quoted in a generated document, a slide, or to a judge as a measured result.

Table of Contents

1. Executive Summary
2. Problem Statement
3. Product Vision & Positioning
4. Product Objectives (measurable)
5. Users, Personas & Jobs-To-Be-Done
6. Scope
7. **Dataset Reality Check & Sizing** ← rewritten in v2.1
8. Functional Requirements
9. System Architecture
10. Technology Stack
11. Data Model & Normalization Contract ← rewritten in v2.1
12. Ingestion Engine Specification
13. Graph Engine Specification
14. Detection Engine Specification ← rewritten in v2.1
15. Mule Risk Index (MRI)
16. Layer Classification Algorithm (L1/L2/L3) ← rewritten in v2.1

17. Money-Trail Tracing Algorithm (4-hop, value-weighted)
18. Frontend / UX Specification
19. API Specification
20. Evidence Object & Anti-Hallucination Architecture
21. Document Generation (Case Diary & S.94 BNSS Freeze Requisition)
22. Non-Functional Requirements
23. Security & Privacy Requirements
24. Evaluation Strategy & Scoring Alignment
25. Test Plan & Acceptance Matrix
26. Team Responsibilities & RACI
27. 36-Hour Execution Plan
28. MVP Ladder, Risks & Contingencies
29. Open Questions & Standing Assumptions
30. Demo Script & Submission Package
31. Appendices
32. **Changelog v2.0 → v2.1** ← new
33. **What Changed Because We Now Have the Real Dataset** ← new

---

1. Executive Summary

Operation “Abhedya-Chakra” is a locally deployable cyber-financial investigation and digital-forensics platform that helps law-enforcement investigators identify and trace **multi-tier money-mule networks** hidden inside large-scale banking transaction data.

The platform ingests and indexes the supplied **2,000,000-row banking transaction dataset** (≈24,873 accounts over ≈15 days — §7), detects layered mule structures using graph analytics and behavioural heuristics, traces stolen funds from a victim account through multiple mule layers, visualises the resulting network interactively, and generates **evidence-grounded case documentation** — a police case diary and a bank freeze requisition under **Section 94 BNSS (erstwhile Section 91 CrPC)**.

The product is built around the three bottlenecks named in the problem statement:

#	Bottleneck	Our answer
1	**Computational scale** — millions of rows, spreadsheets die, SQL multi-hop is slow	Columnar ingestion (Polars/DuckDB) + integer-encoded **CSR adjacency** held in RAM → traversal is array arithmetic, not SQL joins
2	**Layering & smurfing** — collector → distributor → cash-out	Dual-sided feature engine (velocity, fan-in **and** fan-out, matched pass-through, terminal evidence) + **victim-anchored, value-attributed traversal** for layer assignment. Roles are descriptors, not capabilities (§2.4)
3	**Evidentiary bottleneck** — notices must be produced in minutes, not days	**Deterministic templates filled from a validated Evidence Object**, with the LLM restricted to narrative prose only — zero invented facts, structurally guaranteed

The one-sentence pitch: Type a victim's account number, press Enter, and in under two seconds get the complete money trail, the mule hierarchy, the risk scores, the graph, and a printable freeze notice.

New in v2.1: the document is now grounded in the actual dataset. §7 is a Dataset Reality Check, §32 is the changelog, and §33 lists the eight implementation consequences of the real data. Every dataset figure is marked **[DATA-PRELIM]** until the profiler verifies it.

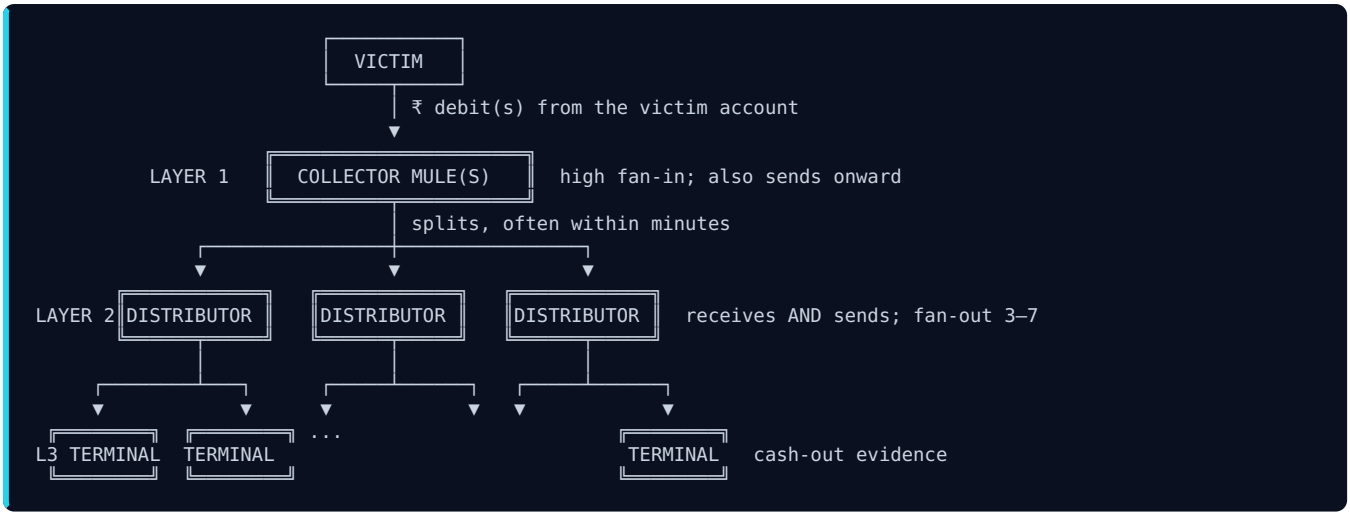
What a judge will see in 5 minutes: blind victim ID → sub-2-second trace → three colour-coded layers on a live graph → timeline scrub → click a terminal node → generate case diary + S.94 BNSS notice as PDF, with every account number and amount traceable to a row ID in the source dataset.

---

2. Problem Statement

Financial cybercrime — digital arrest scams, fake-task/part-time-job schemes, Ponzi apps, and predatory loan apps — routes stolen money through networks of intermediary "mule" accounts that disperse funds within minutes of receiving them. By the time an investigator manually reconciles bank statements, the money has been cashed out.

The laundering topology specified by the problem statement: **[PS]**



2.1 Computational scale [PS]

Bulk transaction exports contain millions of records. Spreadsheets fail outright; naïve relational multi-hop queries (4 self-joins over 2M rows) are too slow for an interactive investigation.

2.2 Layering and smurfing [PS]

Funds are fragmented across Layer-1 collectors, Layer-2 distributors (3–7 downstream accounts each), and Layer-3 terminal cash-out nodes, often inside a 3–15 minute window.

2.3 Evidentiary bottleneck [PS]

Investigators must issue bank freezing notices immediately to prevent dissipation. Drafting these by hand, correctly, with exact account numbers, IFSC codes and transaction IDs, is slow and error-prone.

2.4 Conceptual correction: layers are ROLES, not transaction capabilities [DEC — new in v2.1]

This is the single most important interpretive change in v2.1. It must be reflected in the detection engine, the layer classifier, the MRI explanations, the node inspector and the generated documents.

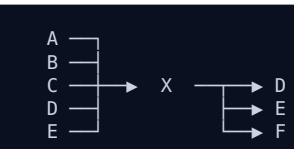
A naïve reading of the topology produces a wrong model:

```
✘ WRONG                                  ✔ CORRECT
─────────────────────────────────────    ──────────────────────────────────────
Collector  = only receives money         Collector  = receives from MANY, and
Distributor= only sends money                         typically forwards onward
Terminal   = has no outgoing at all      Distributor= receives from upstream AND
                                                      splits onward to 3–7 accounts
                                         Terminal   = the apparent END of the trail,
                                                      established by EVIDENCE — not
                                                      merely by "has no outgoing row"
```

Every mule account can both receive and send. An account may simultaneously exhibit high in-degree and high out-degree; the detection engine must therefore compute **both sides of the flow for every account, always** (§14.1). Role is decided by the combination of:

1. **Victim-relative position** in the traced trail (hop depth)
2. **Dominant graph topology** (is fan-in or fan-out the stronger signal?)
3. **Temporal money-flow behaviour** (how fast do received funds leave?)
4. **Upstream/downstream relationships** (who funded it; how many it funded)
5. **Terminal evidence** (cash-out indicators, sink behaviour)

Worked illustration (illustrative, not from the dataset)




```
X has in-degree 5 AND out-degree 3.
X is BOTH collecting and distributing.
In an investigation of a victim who paid A, X is most likely an L1 COLLECTOR,
because its dominant role in THAT trail is aggregation of many inbound senders.
In an investigation of a victim who paid X directly, X is still L1 — but if the
victim paid C, and C paid X, then X is at hop 2 and may be classified L2.
```

2.5 Layers remain victim-relative [DEC — preserved from v2.0]

A layer label is a property of **(account, victim case)**, not of the account alone. The same account can be L1 in one investigation and L2 or L3 in another. The system therefore stores:

- a **case-scoped layer** — computed per trace, used in the graph, case diary and freeze requisition; and
- a **global modal role** — the role the account most often occupies across all analysed cases, used for ranking and for the Ring Board, and retained in case the evaluation interprets layers as global per-account labels (§29 Q10).

---

3. Product Vision & Positioning

> **Build a high-throughput, local-first investigation platform that converts millions of raw banking transactions into an explainable money trail, a ranked mule network, and court-ready investigative output — in seconds.**

The platform moves the investigator from:

`Victim Account ID` → `Money Trail → Mule Layers → Risk Assessment → Evidence → Case Documentation`

without manually searching millions of transactions.

3.1 Design principles (these resolve every design argument during the hackathon)

#	Principle	Consequence
P1	**The database is the only source of truth.**	The LLM never emits a fact. Facts come from SQL; the LLM writes prose around them.
P2	**Precompute everything that the 40%-weighted blind test touches.**	Ingest-time cost is acceptable (60 s budget); query-time cost is not (2 s budget).
P3	**Explainability beats cleverness.**	Every risk score ships with a human-readable reason list. A black-box score we can't explain to a judge is worth less than a transparent heuristic.
P4	**Render the subgraph, never the dataset.**	The browser only ever sees ≤ ~2,000 elements.
P5	**Local-first, no network dependency on the critical path.**	Demo must run with Wi-Fi switched off.
P6	**Demo-path reliability > feature count.**	Any feature not on the demo path is cut first under time pressure.
P7	**Roles are descriptors, not capabilities.** (new in v2.1)	Collector, distributor and terminal all receive **and** send. Compute both directions for every account; decide role from position + dominant topology + behaviour + evidence (§2.4, §16).
P8	**Profile before you assume.** (new in v2.1)	No threshold, enum, schema rule or sizing decision is hard-coded before the profiler has measured it on the actual file (FR-A05). Assumptions carried from v2.0 that the data contradicts are bugs, not opinions.
P9	**Never destroy a source row.** (new in v2.1)	Duplicate `Transaction_ID`s exist in the supplied CSV. Every row is preserved, addressable and traceable; nothing is silently dropped or de-duplicated (§11.3).
P10	**A signal is evidence, not a verdict.** (new in v2.1)	No single feature — not a foreign IP, not `Linux_Script`, not high volume — may classify an account as a mule on its own. Risk is always a weighted combination with a stated reason (§14.4, §15.2).

3.2 Non-goals (explicit)

We are **not** building a bank integration, a real freeze actuator, a live-stream ingester, a police-database connector, or an automated e-filing system. The deliverable is an **investigator's workbench** that produces documents a human officer signs and sends.

4. Product Objectives (measurable)

ID	Objective	Metric	Target	Source
O1	High-throughput ingestion	Wall-clock to ingest + index 2M rows on 16 GB RAM	**≤ 60 s**	[PS]
O2	Mule-network detection	Layer-wise detection of L1/L2/L3 structures	Working on all blind victims	[PS]
O3	Risk scoring	Mule Risk Index per account	**0–100**, explainable	[PS]
O4	Multi-hop tracing	Victim → downstream trail depth	**4 hops in ≤ 2 s** (p95)	[PS]
O5	Interactive investigation	Graph interaction without freezing	**500+ nodes / 1,500+ edges @ ≥30 fps idle**	[PS]
O6	Temporal investigation	Timeline filter across 15-day window	Minute-level granularity, re-filter ≤ 150 ms	[PS]
O7	Automated documentation	Case diary + freeze requisition	Generated ≤ 10 s, printable PDF	[PS]
O8	Anti-hallucination	Invented accounts/amounts/txn IDs in output	**Exactly 0**, programmatically enforced	[PS]
O9	Detection quality	Precision / Recall / F1 on labelled mules	**F1 ≥ 0.85** (stretch 0.90)	[DEC]
O10	Deployment simplicity	Cold clone → running app	**≤ 3 commands, ≤ 5 min**	[DEC]

5. Users, Personas & Jobs-To-Be-Done

5.1 Primary — 🕵 Cybercrime Investigator ("Inspector Rane")

Context: Receives an NCRP/1930 complaint. Has a victim account number, an approximate time, and a bulk CSV export from the bank's nodal officer. Under a golden-hour clock — funds move out of the ecosystem within hours.

Jobs: - "Given this victim account, where did the money actually go?" - "Which accounts are still holding my victim's money right now?" - "Which 5 accounts should I ask the bank to freeze first?" - "Show me, visually, that account X is the hub of this ring." - "Give me a printable notice with correct account numbers and IFSC codes."

Success for this user: Under 5 minutes from receiving the complaint to a signed freeze requisition going out to the bank.

Pain we must not reproduce: Tools that require SQL knowledge, tools that return 10,000 rows of CSV, tools that hang.

5.2 Secondary — 👨‍⚖️ Investigative / Legal Personnel ("PP / Court")

Needs chronological evidence, exact transaction identifiers, amounts, timestamps and account details in a structured, dated case diary that can withstand cross-examination. **Every claim must be reproducible from the source dataset.**

5.3 Secondary — 🏦 Bank / Nodal Officer

Needs an unambiguous, machine-actionable list: beneficiary account number, IFSC, disputed transaction IDs, amounts and the lien/debit-freeze request — in the format they already process.

5.4 Tertiary — 👨‍⚖️ Hackathon Judge (be honest: this is a real persona)

Needs to type an unknown account ID and see something correct, fast and visually legible, then see a document that looks like it came out of a police station. **Optimise the first 90 seconds of the demo for this persona.**

---

6. Scope

✔ In scope

Transaction ingestion · data normalization · account indexing · transaction querying · graph construction · mule detection · risk scoring · 4-hop tracing · layer identification · terminal-node detection · interactive graph · timeline filtering · subgraph isolation · case-diary generation · freeze-requisition generation · performance benchmarking · offline/local core execution.

✘ Out of scope (explicitly, and we say so in the demo)

Direct banking-system integration · real-world account freezing · live transaction ingestion · police-database integration · automated legal filing · any actual financial intervention · user authentication/multi-tenancy · cloud deployment · mobile app · KYC/identity resolution of account holders.

● Deferred (post-hackathon roadmap slide)

GNN-based detection · cross-bank entity resolution · real NCRP/1930 API hookup · multi-case management · audit-logged chain of custody with digital signatures.

7. Dataset Reality Check & Sizing

v2.1 replaces the estimated sizing of v2.0 with figures taken from the supplied file. Read this section before writing any code — three v2.0 defaults are wrong against this data and are corrected here.

7.0 Verification status — read this first ⚠

Source of the figures below	Preliminary profiling reported by the team
Independently re-verified against the CSV by the PRD author?	**NO — the file was not available when v2.1 was written**
How they become verified	`make profile` (FR-A05) writes `data/artifacts/profile.json`; `make profile-check` diffs it against the table in §7.2 and **fails loudly on any mismatch**
Until then	Every figure is tagged **[DATA-PRELIM]**. Do not quote them on a slide, in a README benchmark table, or to a judge as measured results.

First task of Phase 1 is to run the profiler and replace this status block with `✔ VERIFIED — profile.json sha256:… on <date>`. Any figure the profiler contradicts is corrected here, and §33 is updated with the consequence.

What was checked, by arithmetic, without the file:

Check	Result
Payment-mode counts sum to row count	1,299,844 + 441,199 + 199,134 + 59,823 = **2,000,000** ✔
Device-type counts sum to row count	666,004 + 665,917 + 665,425 + 1,327 + 1,327 = **2,000,000** ✔
Row count − unique `Transaction_ID`	2,000,000 − 1,997,748 = **2,252** ✔ consistent with the reported duplicate figure
Automation devices vs foreign-IP rows	`Web_Emulator` + `Linux_Script` = 2,654 **= exactly** the foreign-IP row count ✔
Crypto + wallet narration rows	1,327 + 1,327 = 2,654 **= the same total** ✔

The internal consistency is strong, which raises confidence — but consistency is not verification. See §7.4 for why the last two rows are the most consequential finding in this document.

7.1 Observed dataset profile [DATA-PRELIM]

Property	Value	Note
File	`VoidHacks8_MuleAccount_2M_Transactions.csv`	≈274 MB
Rows	**2,000,000**	exactly — not "2,000,000+"
Columns	11	matches the PS schema
Date range	**2026-09-15 → 2026-09-29**	≈15 days, consistent with [PS]
Unique accounts	**≈24,873**	the graph population
Unique `Transaction_ID`	**≈1,997,748**	**not unique across rows** — see §7.3

Property	Value	Note
Missing values	none observed	re-confirm per column in the profiler
Amount min / median / mean / max	₹50 / **₹899.35** / ₹1,760.88 / ₹499,823.99	right-skewed, mean/median ≈ 1.96
Payment modes	UPI 1,299,844 · IMPS 441,199 · NEFT 199,134 · RTGS 59,823	UPI = 65.0%
Device types	Windows_Browser 666,004 · iOS 665,917 · Android 665,425 · **Web_Emulator 1,327 · Linux_Script 1,327**	three "normal" devices are near-perfectly balanced at ~33.3% each
Foreign-IP rows (`185.*`, `194.*`)	≈2,654	0.1327% of rows
Automation-device rows	≈2,654	0.1327% of rows
P2P/crypto narration rows	≈1,327	0.0664%
Wallet narration rows	≈1,327	0.0664%

Derived quantities (arithmetic from the above — recompute in the profiler)

Quantity	Value	Why it matters
Transactions per day	≈133,333	timeline histogram bucket sizing
Avg transactions per account **as sender**	**≈80.4**	this is the out-degree, and it is large
Avg transactions per account **as receiver**	≈80.4	in-degree equally large
Avg total degree	≈160.8	—
Avg transactions per account per day	≈5.4	drives the traversal window choice (§7.5)
Expected **unique (sender→receiver) pairs**	**≈2.0 M**	§7.6 — v2.0 assumed 300k–900k and was wrong

7.2 Profiler expectation table (machine-checked)

`make profile-check` asserts each of these. A mismatch is a **P0 finding**, not a rounding issue.

Key	Expected	Tolerance	Actual (fill from `profile.json`)	✔
`row_count`	2,000,000	exact		
`column_count`	11	exact		
`unique_accounts`	24,873	±1%		
`unique_txn_ids`	1,997,748	±0.1%		
`txn_id_row_surplus`	2,252	exact		
`exact_duplicate_rows`	unknown	—		
`date_min` / `date_max`	2026-09-15 / 2026-09-29	exact day		
`amount_median`	899.35	±1%		
`amount_max`	499,823.99	±1%		
`mode_counts.UPI`	1,299,844	±0.1%		

Key	Expected	Tolerance	Actual (fill from `profile.json`)	✔
`device_counts.Web_Emulator`	1,327	exact		
`device_counts.Linux_Script`	1,327	exact		
`foreign_ip_rows`	2,654	exact		
`narr_crypto_rows`	1,327	exact		
`narr_wallet_rows`	1,327	exact		
`unique_pairs`	≈1,996,771	±5%		
`max_out_degree` (hub ceiling)	unknown	—		
`signal_cooccurrence` (§7.4)	unknown	—		

7.3 `Transaction_ID` is not a primary key [DATA-PRELIM]

2,000,000 rows carry ≈1,997,748 distinct `Transaction_ID` values — a surplus of **2,252 rows**.

> ⚠ **The reported "2,252 duplicate rows" is the row surplus, not the number of rows involved.** If every duplicated ID appears exactly twice, then 2,252 IDs are duplicated and **4,504 rows** are affected. If one ID appeared 2,253 times, 2,253 rows would be affected. **The profiler must report the ID-repeat histogram** (`{repeat_count: n_ids}`) so this is resolved with a number, not an inference.

Consequences, specified in full in §11.3: we introduce an internal `source_row_id` (the 0-based CSV row ordinal) as the immutable provenance anchor and primary key; `Transaction_ID` becomes a source-provided identifier that is indexed but not assumed unique; and **no duplicate row is ever deleted or quarantined**.

7.4 ⚠ The terminal signals are a single injected composite, not four features [DATA-PRELIM]

Three independent counts land on the same number:

```
foreign-IP rows (185.* , 194.*)      ≈ 2,654
automation-device rows               ≈ 2,654     ( Web_Emulator 1,327 + Linux_Script 1,327 )
crypto narration + wallet narration  ≈ 2,654     ( 1,327 + 1,327 )
```

The most probable explanation is that the generator injected **one set of ~2,654 "terminal" transactions**, each carrying a foreign IP **and** an automation device **and** one of {crypto, wallet} narration. **[ASSUME]**

Two consequences, both significant:

1. **Do not score these as four independent pieces of evidence.** MRI component C5 in v2.0 blends four terminal sub-signals; if they always co-occur, that blend quadruple-counts one fact and lets a single injected flag dominate the score — violating principle P10. §15.1 therefore redefines C5 as a **composite with a co-occurrence discount**, and the profiler must emit a **cross-tabulation** (`signal_cooccurrence`) so M3 can confirm or refute the assumption in Phase 1.
2. **Terminal signals cannot be the primary detection mechanism.** They touch 0.13% of rows and at most ~5,308 account-endpoints. Against the PS-described population of 1,500 mule accounts, they plausibly mark only the **L3 subset** — perhaps a few hundred accounts at ~5 signal transactions each. **[ASSUME]**

```
RECALL CEILING FROM TERMINAL SIGNALS ALONE
──────────────────────────────────────────────────────────────
L1 collectors  : no terminal signal expected  → 0% found this way
L2 distributors: no terminal signal expected  → 0% found this way
L3 terminals   : signal-bearing              → found this way
⇒ Terminal signals are a PRECISION booster for L3.
  Recall across all three layers MUST come from topology + behaviour.
```

Build consequence: M3's time goes into fan-in/fan-out, pass-through and velocity features — the things that find L1 and L2 — not into enriching the terminal keyword list. The keyword list is already complete; it just does not cover two-thirds of the problem.

7.5 ⚠ Branch explosion: the v2.0 traversal window saturates the graph [DATA-PRELIM]

With ≈80.4 outgoing transactions per account over ≈15 days, the expected branching factor inside a traversal window of H hours is `b ≈ 80.4 × H / 360`:

Window	b	hop 1	hop 2	hop 3	hop 4	Verdict
72 h (v2.0 default)	16.1	16	259	4,159	**66,885**	✘ exceeds the 24,873-account population — **the trace returns the entire graph**
24 h	5.4	5	29	154	826	⚠ at the UI node cap; acceptable with pruning
6 h (new default)	1.34	1.3	1.8	2.4	3.2	✔ tight, and comfortably covers a 3–15 minute laundering chain
1 h	0.22	—	—	—	—	too tight; risks truncating slower chains

`TRACE_WINDOW_SEC` **default changes from 72 h to 6 h** (§17.2, Appendix B), exposed as a UI control with 1 h / 6 h / 24 h / 72 h presets so an investigator can widen it deliberately and see the cost. The PS pattern completes in 3–15 minutes, so 6 hours is already a 24× margin. **[DEC]**

> This finding alone justifies the value-attributed, temporally-ordered traversal design of v2.0 (§17). On this dataset a plain 4-hop BFS does not merely get slow — it returns everything, which is the same as returning nothing.

7.6 ⚠ The edge count is ~2 M, not ~900 k — recompute the memory budget [DATA-PRELIM]

v2.0 assumed transactions would concentrate onto 300k–900k repeated account pairs. With 24,873 accounts there are ≈618.6 M possible directed pairs and only 2 M transactions, so collisions are rare: the expected number of **unique pairs is ≈1,996,771 — about 99.8% of rows are their own unique pair**.

```
v2.0 estimate :  E ≈ 0.9 M  →  ~26 MB per direction
v2.1 estimate :  E ≈ 2.0 M  →  ~64 MB per direction  →  ~128 MB both   [DATA-PRELIM]
```

Still entirely acceptable — it is ~128 MB inside a multi-GB budget — but the team must not be surprised by it, and edge aggregation buys us almost nothing on this data. The CSR build therefore keeps per-pair aggregation (it is cheap and still correct) but sizes arrays for `E ≈ rows`. The profiler reports `unique_pairs` so this is measured, not assumed.

7.7 The core sizing insight — unchanged and now confirmed

```
TRANSACTION ROWS : 2,000,000     ← the big thing: a columnar fact table
UNIQUE ACCOUNTS  :    24,873     ← the small thing: an in-RAM integer graph
AVG TOTAL DEGREE :      ~161
```

The architecture of v2.0 stands:

1. **Separate the log from the graph.** A large immutable columnar fact table (DuckDB/Parquet) for evidence; a small integer-encoded CSR adjacency (NumPy) for traversal; joined by a dense `int32` account index.
2. **Per-account work is cheap.** 24,873 rows of features is nothing — the entire detection pass is one `GROUP BY` plus vectorised arithmetic at ingest time.
3. **Do not use NetworkX in the query hot path. [DEC]** Python object graphs at 24,873 nodes × 2 M edges cost hundreds of MB and are far slower than CSR + NumPy. NetworkX is permitted only for offline sanity checks on sampled subgraphs.
4. **The real risk is branch explosion, not data volume** (§7.5). Pruning, temporal causality and the hub guard are load-bearing, not optional.

> **Rule of thumb, unchanged:** if a query touches 2 M rows at request time, it's wrong. Pre-aggregate it at ingest.

---

8. Functional Requirements

Module A — High-Throughput Ingestion & Normalization

FR-A01 · Large Dataset Ingestion [PS]

The platform **shall** ingest the supplied 2,000,000-row dataset without out-of-memory failure, using memory-efficient columnar technology (DuckDB / Polars / Arrow). **Acceptance:** `make ingest` completes in ≤60 s, peak RSS ≤ 6 GB, prints a per-stage timing table, and asserts **`rows_out == rows_in == 2,000,000`** — no silent row loss, including duplicates (§11.3).

FR-A02 · Transaction Normalization [PS]

Normalize: bank accounts (12-digit, zero-padded **string**), IFSC (upper-cased, validated), payment mode (enum from the observed set), IP (string + `/8` prefix + foreign flag), narration (lower-cased + keyword flags), device type (enum + automation flag), timestamp. **Changed in v2.1:** enum domains are **discovered by the profiler, not hard-coded** (P8). The observed domains — modes {UPI, IMPS, NEFT, RTGS}, devices {Windows_Browser, iOS, Android, Web_Emulator, Linux_Script} **[DATA-PRELIM]** — become the expected set; any unexpected value is counted and surfaced, never silently mapped to `OTHER`. **Acceptance:** normalization report lists counts per enum value, null counts per column, and rows flagged malformed. **Target: 0 rows dropped.** Malformed rows are flagged in place, not removed.

FR-A03 · Account Search [PS]

Search an account and retrieve historical transactions, counterparties, timeline, incoming and outgoing transactions, at sub-second speed. With ≈80 incoming and ≈80 outgoing transactions per account on average **[DATA-PRELIM]**, the default un-paginated view is comfortably small; pagination engages beyond 500 rows. **Acceptance:** p95 ≤ 300 ms.

FR-A04 · Dataset Fingerprint & Reproducibility [DEC]

On ingest, record a manifest: file SHA-256, row count, min/max timestamp, distinct account count, **distinct `Transaction_ID` count, duplicate-ID row count, exact-duplicate row count**, schema hash, ingest duration, tool versions, and the `config.py` snapshot. Displayed in the UI footer and embedded in every generated document.

FR-A05 · Dataset Profiling & Schema Discovery [DEC — new in v2.1]

On first ingest (and on demand via `make profile`), the system **shall** profile the file before any analytical assumption is applied, and write the result to `data/artifacts/profile.json`, summarised in `manifest.json` and rendered on the Ingest/System screen.

Required profile contents:

Group	Items
Shape	row count · column count · file size · column names and inferred dtypes · schema hash
Time	min / max timestamp · span in days · per-day row histogram · per-hour-of-day histogram
Identity	unique accounts · unique `Transaction_ID` · **ID-repeat histogram** `{repeat_count → n_ids}` · rows affected by duplicate IDs · **exact-duplicate row count** (all 11 fields equal) · near-duplicate count (same ID, differing fields)
Completeness	null / empty count per column · malformed IFSC count · malformed IP count · non-12-digit account count
Amounts	min · p1 · p25 · **median** · mean · p75 · p99 · max · zero/negative count · round-amount share · amount histogram (log bins) · **amount × payment-mode cross-tab**

Group	Items
Categorical	payment-mode distribution · device-type distribution · narration keyword distribution · IP first-octet distribution · IFSC-prefix (bank) distribution
Graph shape	unique (sender→receiver) pairs · in-degree and out-degree distributions (p50/p90/p99/max) · **max out-degree (hub ceiling)** · self-loop count · reciprocal-pair count · isolated-account count
Signals	foreign-IP rows · automation-device rows · crypto-narration rows · wallet-narration rows · `signal_cooccurrence` **cross-tab** (§7.4) · accounts touched by ≥1 signal transaction
Derived	avg txns per account (in / out) · estimated branching factor for 1 h / 6 h / 24 h / 72 h windows (§7.5)

Acceptance: `make profile` runs in ≤90 s and `make profile-check` compares `profile.json` against the expectation table in §7.2, exiting non-zero on any out-of-tolerance value. **The system must not hard-code an assumption that the profiler has not confirmed.**

Module B — Mule Ring Detection & Graph Analytics

FR-B01 · Transaction Graph [PS]

Nodes = accounts; edges = transactions (directed, timestamped, amount-weighted). Parallel edges between a pair are aggregated for traversal but **every individual source row is preserved for evidence** (§11.3). On this dataset ≈99.8% of rows are their own unique pair **[DATA-PRELIM]**, so aggregation is near-identity.

FR-B02 · Dual-Sided Account Features [DEC — new in v2.1, promoted from §14]

For **every** account the engine **shall** compute the incoming profile, the outgoing profile and the cross-flow relationship between them (full catalogue in §14.1). An account is never characterised from one side of the ledger alone (P7).

FR-B03 · Mule Risk Index [PS]

Every analyzed account receives a **0–100** index with a per-component breakdown and a human-readable reason list. No single component may, by itself, place an account in the CRITICAL band (§15.2). Formula in §15.

FR-B04 · High-Velocity Pass-Through [PS]

Flag accounts where **≥90% of incoming funds** are dispersed via outgoing transfers within **3–15 minutes** across multiple outgoing transactions. v2.1 replaces the crude `total_out/total_in` ratio with **temporal matching of outgoing value to specific incoming credits** (§14.2). Thresholds remain configurable; a widened secondary window is scored at reduced weight so near-misses are not lost.

FR-B05 · Collector-Role Detection (Layer 1) [PS]

Identify L1 collectors via **high fan-in** — many distinct inbound senders, high in-degree relative to out-degree — **combined with** rapid onward dispersion of the collected funds. **A collector normally also sends money; "receives money" alone is not the signal** (§2.4).

FR-B06 · Distributor-Role Detection (Layer 2) [PS]

Identify L2 distributors via **high fan-out** — approximately **3–7 downstream recipients** — **combined with** receipt from an upstream mule-like account, splitting behaviour (consistent sibling amounts) and rapid onward movement. **A distributor necessarily also receives money; "sends money" alone is not the signal** (§2.4).

FR-B07 · Terminal-Role Detection (Layer 3) [PS]

Identify L3 candidates from **cash-out evidence**: wallet or P2P/crypto narration, foreign IP headers (`185.*`, `194.*`), automation/headless device profiles (`Web_Emulator`, `Linux_Script`), and/or strong sink behaviour (little or no onward transfer of received value). **Changed in v2.1:** (a) these indicators are **weighted evidence, never proof** (P10); (b) they are treated as a **co-occurring composite**, not four independent signals (§7.4); (c) **an account with zero outgoing rows is not automatically terminal**, and an account with some outgoing activity is not automatically non-terminal — terminal status requires evidence, of which "funds stop here" is one form.

FR-B08 · Multi-Hop Trace [PS]

Given a victim account, trace the downstream money trail up to **4 hops** with p95 ≤ **2 s**. Algorithm and the recalibrated window/pruning defaults in §17.

FR-B09 · Ring / Community Detection [DEC]

Identify connected mule rings (weakly-connected components of the high-risk subgraph) and rank them by value moved. Supports the detection criterion independently of any specific victim.

FR-B10 · Cycle & Return-Path Detection [DEC]

Detect funds returning to an earlier node in the trail, up to length 4, and flag them as cycle participation.

Module C — Interactive Law-Enforcement Graph

FR-C01 · Graph Visualization [PS]

Interactive graph showing accounts, transactions, flow direction, amounts, timestamps, suspicious nodes and mule layers. Technology: Cytoscape.js **[DEC]**.

FR-C02 · Clustered Subgraph Rendering [PS]

Never render the full dataset; render the relevant suspect subgraph, responsive at **500+ nodes / 1,500+ edges**. Beyond `GRAPH_NODE_CAP` (default 800) low-value leaves collapse into an expandable `+N accounts (₹X)` node. Given the branching factor measured in §7.5, **the cap will actually be hit** on wide windows — the collapse path is a core feature, not a safety net.

FR-C03 · Temporal Playback [PS]

Filter across the ≈15-day window (**2026-09-15 → 2026-09-29 [DATA-PRELIM]**) with minute-level granularity, plus animated playback. Day boundaries and the histogram are built from the profiler's per-day counts (≈133,333 rows/day), not from hard-coded dates.

FR-C04 · Subgraph Isolation & Export [PS]

Clicking an account isolates its network; the isolated subgraph exports as a sub-dataset (CSV of the underlying **source rows, including** `source_row_id`, JSON of the graph, PNG of the view).

FR-C05 · Node Inspector [DEC]

Right panel showing account number, IFSC, bank, MRI with component bars, reason list, **both-sided flow summary (in: count/amount/distinct senders — out: count/amount/distinct receivers)**, **role badge with the explicit statement that the role is case-relative**, layer, confidence, first/last seen, counterparties, and the raw transaction table with `source_row_id` and one-click "add to evidence basket".

FR-C06 · Evidence Basket [DEC]

Pin accounts/transactions into a basket; documents are generated from the basket (auto-populated from the trace by default).

Module D — Local AI Case Officer

FR-D01 · Case Diary Generation [PS]

Generate a structured chronological case diary containing, where available from verified data: victim account, total funds siphoned, Layer-1/2/3 accounts, timestamps, exact transaction amounts, current holding accounts, and accounts recommended for immediate freezing. **Each listed account states its role, its layer, its confidence and the evidence for both.**

FR-D02 · Bank Freeze Requisition [PS]

Generate a pre-formatted requisition per beneficiary bank containing account number, IFSC, disputed transaction ID(s), amounts and transaction details, in the **Section 94 BNSS (erstwhile Section 91 CrPC)** idiom. **Changed in v2.1:** because `Transaction_ID` is not unique (§7.3), any disputed transaction whose ID is shared by more than one source row **must** be disambiguated in the document by date-time + amount + counterparty, and internally by `source_row_id`.

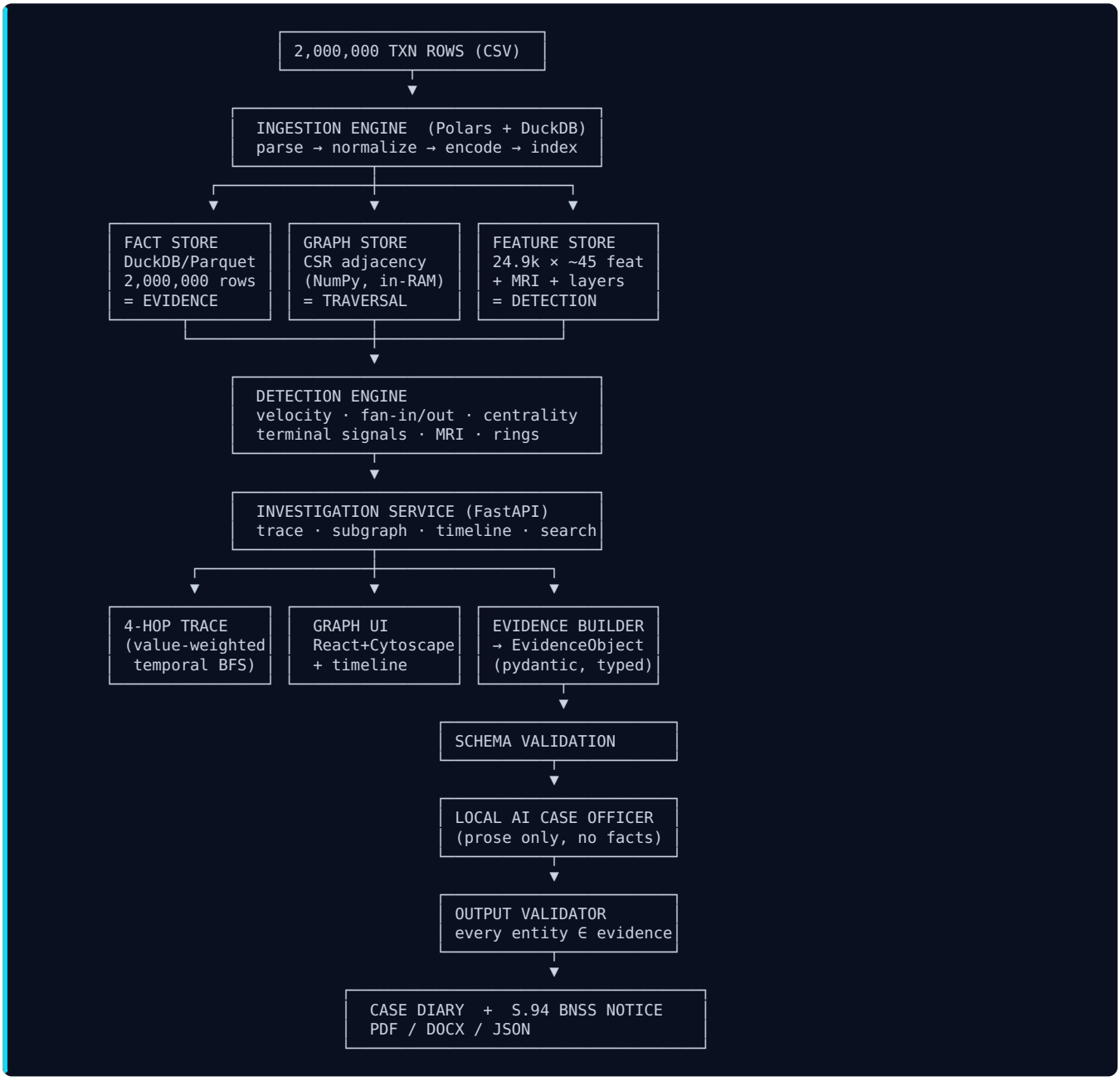
FR-D03 · Evidence-Grounded AI [PS]

The AI shall not invent investigative facts. Account numbers, amounts and transaction IDs must originate from verified system data, enforced programmatically (§20).

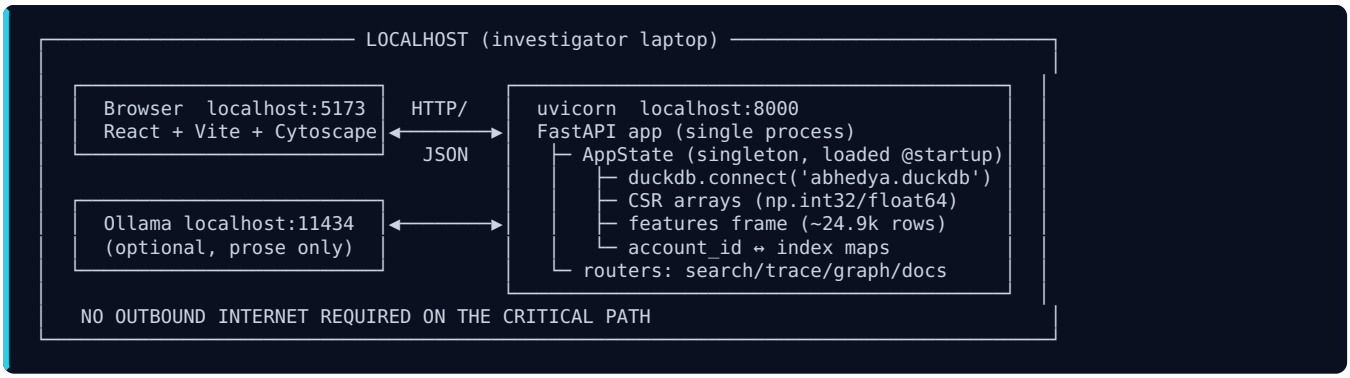
---

9. System Architecture

9.1 High-level



9.2 Process & deployment view [DEC]



Single-process rationale: the graph lives in RAM; multiple workers would duplicate it (and 2× the memory). We run `uvicorn --workers 1` with async endpoints and offload CPU-bound traces to a small thread pool (NumPy releases the GIL). **[DEC]**

9.3 Repository layout [DEC]

```
abhedya-chakra/
├── README.md                    # setup in 3 commands + screenshots + benchmarks
├── Makefile                     # make setup | ingest | run | bench | test | demo
├── pyproject.toml
├── data/
│   ├── raw/VoidHacks8_..._2M_Transactions.csv   # gitignored (~274 MB)
│   └── artifacts/               # abhedya.duckdb, graph.npz, features.parquet,
│                                # profile.json, manifest.json
├── backend/
│   └── abhedya/
│       ├── main.py              # FastAPI app factory + lifespan loader
│       ├── config.py            # ALL thresholds, one place, env-overridable
│       ├── state.py             # AppState singleton
│       ├── ingest/              # loader.py, normalize.py, encode.py, manifest.py,
│       │                        # profile.py (FR-A05), duplicates.py
│       ├── graph/               # csr.py, traverse.py, subgraph.py
│       ├── detect/              # features.py (dual-sided), passthrough.py (FIFO),
│       │                        # rules.py, scoring.py, layers.py, rings.py
│       ├── evidence/            # builder.py, schema.py, validator.py
│       ├── docs/                # diary.py, requisition.py, pdf.py, templates/*.j2
│       ├── api/                 # routers/*.py
│       └── bench/               # bench_ingest.py, bench_trace.py, bench_eval.py
├── frontend/
│   └── src/
│       ├── App.tsx
│       ├── api/client.ts
│       ├── components/  (SearchBar, KpiStrip, GraphCanvas, TimelineSlider,
│       │                 NodeInspector, EvidenceBasket, DocumentModal, LayerLegend)
│       ├── hooks/       (useTrace, useTimeline, useGraphLayout)
│       └── styles/
├── tests/                       # pytest: unit + golden-file + perf smoke
└── docs/                        # PRD.md, ARCHITECTURE.md, EVALUATION.md, DEMO.md, slides/
```

10. Technology Stack

Layer	Choice	Why this, not the alternative
Data processing	**Polars**	Multi-threaded, lazy, Arrow-native; 5–20× pandas on 2M rows; strict schema
Analytical DB	**DuckDB**	Zero-config embedded OLAP; vectorised CSV reader; SQL for evidence pulls; single-file artifact
Graph structures	**NumPy CSR** (hand-rolled)	≈24.9k nodes / ≈2.0 M edges ≈ 128 MB both directions (§13.1); traversal in compiled code. NetworkX only for offline validation
Backend	**Python 3.11 + FastAPI + uvicorn**	Async, typed, auto-OpenAPI (free API docs for judges), pydantic validation on the evidence path
Frontend	**React 18 + Vite + TypeScript**	Fast HMR during a 36-h build; TS prevents 3 a.m. prop bugs
Graph viz	**Cytoscape.js** (`react-cytoscapejs`)	Best-in-class for directed, labelled, compound graphs; built-in layouts (`dagre` for layered flow, `fcose` for rings); handles 2k elements comfortably
Layout	**dagre** (layered L-R) + **fcose** (force) toggle	Dagre makes Victim→L1→L2→L3 read like the PS diagram — huge for judging
Charts	**Recharts**	Small, declarative, enough for MRI breakdown + timeline histogram
Styling	**Tailwind CSS** + custom "operations console" dark theme	Speed; no CSS bikeshedding at 4 a.m.
Local AI	**Ollama** (`llama3.1:8b-instruct` or `qwen2.5:7b-instruct`), prose-only	Runs offline; satisfies local-execution requirement. Degrades to pure template if absent
Documents	**Jinja2 → HTML → WeasyPrint** (PDF); **python-docx** (DOCX)	Exact typographic control, Devanagari-capable fonts, deterministic output
Testing	**pytest** + **pytest-benchmark**	Perf regressions are a scored criterion
Packaging	**Makefile + requirements.txt** (+ optional Dockerfile)	3-command setup; Docker optional so judges aren't forced to pull images
VCS	**Git + GitHub**, conventional commits, PR-per-module	10% engineering score is partly repo hygiene

Hard constraint [PS]

Core ingestion, graph computation and dashboard rendering **must operate locally/offline**. Therefore **no cloud service may sit on the critical path**. The only optional network call is the LLM prose layer, which has a deterministic offline fallback and is disabled in the demo by default.

11. Data Model & Normalization Contract

11.1 Source schema [PS] — with v2.1 normalization rules

Field	Type	Description	Normalization rule
`Transaction_ID`	string	Source-provided identifier	Trim, upper-case. **NOT assumed unique** (§7.3, §11.3). Indexed, never a primary key.
`Sender_Account`	string	12-digit originating account	Strip non-digits, zero-pad to 12, keep as **string** (leading zeros are real)
`Receiver_Account`	string	12-digit beneficiary account	Same
`Sender_IFSC`	string	Sender bank IFSC	Upper-case; validate `^[A-Z]{4}0[A-Z0-9]{6}$` ; bank code = first 4 chars; failures **flagged, not dropped**
`Receiver_IFSC`	string	Receiver bank IFSC	Same
`Amount`	float	Value in INR	`float64` ; observed range ₹50 – ₹499,823.99, median ₹899.35 **[DATA-PRELIM]**; zero/negative flagged, not dropped
`Timestamp`	datetime	Transaction time	Parse to `datetime[us]` ; derive epoch seconds. Observed span 2026-09-15 → 2026-09-29 **[DATA-PRELIM]**
`Payment_Mode`	enum	Observed: UPI, IMPS, NEFT, RTGS	Upper-case → `uint8` code; **unexpected values counted and surfaced**, never silently merged
`Narration`	string	Remarks	Lower-cased token set + keyword flags; **raw text always retained for evidence**
`IP_Address`	string	Client IP	Validate IPv4; derive first octet; foreign flag from a **configurable** prefix set
`Device_Type`	string	Client profile	Observed: Windows_Browser, iOS, Android, Web_Emulator, Linux_Script; automation flag from a **configurable** set

Timezone [ASSUME]: timestamps are naive local time (treated as IST) and are **never converted**. Displayed exactly as supplied. Mentor question §29 Q11 remains open; the fallback costs nothing because no conversion occurs anywhere in the pipeline.

11.2 Derived columns (computed at ingest) [DEC]

Column	Type	Derivation
`source_row_id`	`int64`	**0-based ordinal of the row in the source CSV. Immutable. The primary key and the provenance anchor for all evidence.** (new in v2.1)
`evidence_row_id`	string	`"{dataset_fingerprint_short}:{source_row_id}"` — globally unique across datasets; what appears in documents and exports. (new in v2.1)
`txn_id_is_duplicated`	`bool`	True when this row's `Transaction_ID` occurs on more than one source row. (new in v2.1)
`txn_id_occurrence`	`uint16`	1-based index of this row among rows sharing its `Transaction_ID` , ordered by `source_row_id` . (new in v2.1)
`row_hash`	`uint64`	Hash of all 11 source fields — identifies **exact** duplicates. (new in v2.1)

Column	Type	Derivation
`is_exact_duplicate`	`bool`	True when `row_hash` occurs more than once; the first occurrence is marked `is_duplicate_primary`. (new in v2.1)
`src_idx`, `dst_idx`	`int32`	Dense account index from the global account dictionary
`ts_epoch`	`int64`	Unix seconds
`day_index`	`uint8`	`floor((ts − dataset_start)/86400)` → 0..14
`minute_bucket`	`int32`	`ts_epoch // 60` — timeline filtering key
`is_foreign_ip`	`bool`	First octet ∈ `FOREIGN_IP_PREFIXES` (default {185, 194}, configurable)
`ip_octet1`	`uint8`	For pattern analytics
`is_automation_device`	`bool`	`Device_Type ∈ AUTOMATION_DEVICES` (default {Web_Emulator, Linux_Script}, configurable)
`narr_crypto`	`bool`	Configurable keyword set (p2p, crypto, usdt, btc, …)
`narr_wallet`	`bool`	Configurable keyword set (wallet, topup, …)
`terminal_signal_mask`	`uint8`	**Bitmask** of {foreign_ip, automation, crypto, wallet} — enables the co-occurrence cross-tab of §7.4 instead of four independent booleans. (new in v2.1)
`amount_round`	`bool`	`Amount % 1000 == 0` (structuring hint)
`sender_bank`, `receiver_bank`	`str[4]`	IFSC prefix

> All keyword lists, IP prefixes and device sets live in `config.py` (Appendix B) and are **configurable**, per P8 and Part 8 of the dataset review.

11.3 Duplicate `Transaction_ID` handling — normative [DEC — new in v2.1]

The supplied CSV contains repeated `Transaction_ID` values (§7.3). v2.0 said "assert unique; duplicates quarantined". That instruction is withdrawn and must not be implemented.

Rules

#	Rule
D1	**Every source row is preserved.** Row count out == row count in == 2,000,000. No row is deleted, quarantined, merged or silently de-duplicated at any stage.
D2	`source_row_id` (CSV ordinal) is the **primary key**. All internal joins, graph edges, evidence records and exports key on it.
D3	`Transaction_ID` is a **source-provided identifier**, indexed for lookup, never assumed unique. Any lookup by `Transaction_ID` returns a **list** of rows.
D4	**Near-duplicates** (same ID, differing fields) are both retained and are distinguishable in every surface by `source_row_id`, timestamp, amount and counterparty.
D5	**Exact duplicates** (all 11 fields identical) are retained, flagged `is_exact_duplicate`, and **reported as a distinct profiling category** — they are a property of the evidence, possibly a double-posting artefact, and that is a finding an investigator may need.
D6	Exact duplicates are **counted once** in account-level aggregates used for scoring (`is_duplicate_primary` filter) to avoid inflating volume features, **but both rows remain fully visible** in evidence, exports and the transaction table. The aggregate and the raw view may therefore differ by design — the UI states this where it occurs.

#	Rule
D7	Graph edges carry the **list** of contributing `source_row_id`s. An edge's amount is the sum over its primary rows (D6).
D8	Any document, API response or export that cites a transaction cites it as `Transaction_ID` **plus** date-time, amount and counterparty; where the ID is duplicated, this disambiguation is **mandatory**, and `evidence_row_id` is carried in the machine-readable payload.
D9	The ingest report and `profile.json` state the ID-repeat histogram, affected row count and exact-duplicate count. Silence is not acceptable output.

Why this matters beyond tidiness

A freeze requisition that cites a transaction ID matching two different transfers is **challengeable**. Preserving row-level provenance is what makes the output defensible — and it is cheap: one `int64` column.

11.4 Account dictionary (`accounts` table)

Column	Type	Notes
`account_idx`	`int32` PK	Dense 0..N−1, N ≈ 24,873 **[DATA-PRELIM]**
`account_no`	`varchar(12)`	Unique, string
`primary_ifsc`	`varchar(11)`	Modal IFSC observed
`bank_code`	`varchar(4)`	
`first_seen`, `last_seen`	`timestamp`	
`txn_count_in` / `txn_count_out`	`int32`	**both sides always populated** (P7)
`amount_in` / `amount_out`	`double`	both sides always populated
`distinct_senders` / `distinct_receivers`	`int32`	both sides always populated

11.5 Storage artifacts after ingest

```
data/artifacts/
├── abhedya.duckdb        # transactions (keyed by source_row_id), accounts, features,
│                         # ring_members, duplicate_report
├── graph.npz             # out_* and in_* CSR arrays, E ≈ 2.0 M per direction  [DATA-PRELIM]
├── features.parquet      # ≈24.9k accounts × ~45 features + MRI + components + reasons
├── profile.json          # FR-A05 dataset profile  ← new in v2.1
└── manifest.json         # fingerprint, counts, duplicate stats, timings, config snapshot
```

12. Ingestion Engine Specification

12.1 Pipeline (v2.1 adds stages [0] and [5b]; timings remain estimates until measured)

```
[0] PROFILE     FR-A05 scan: shape, dupes, dists, graph shape    ~10–20 s  (first run / on demand)
      ↓
[1] READ        duckdb.read_csv(parallel) / pl.scan_csv            est. 8–15 s
      ↓
[2] NORMALIZE   typed casts, enum mapping, IFSC/IP validation      est. 5–8 s
      ↓
[3] ENCODE      factorize accounts → int32 dense index             est. 3–5 s
      ↓
[4] DERIVE      source_row_id, row_hash, dup flags, terminal mask,
                day_index, minute_bucket, keyword flags            est. 4–7 s
      ↓
[5] PERSIST     transactions (PK source_row_id) + accounts dim     est. 6–10 s
     [5b] DUPES duplicate_report: ID histogram, exact dupes        est. 1–2 s   ← new in v2.1
      ↓
[6] AGGREGATE   per-account IN and OUT features, single pass       est. 3–5 s
      ↓
[7] BUILD CSR   sort by (src, ts) → indptr/indices; E ≈ 2.0 M      est. 5–8 s   ← resized, §7.6
      ↓
[8] SCORE       MRI + role priors + rings + cycles (≈24.9k rows)   est. 1–2 s
      ↓
[9] MANIFEST    fingerprint + duplicate stats + timings + config   <1 s

                        TOTAL  —  TARGET ≤ 60 s [PS]  ·  ACTUAL: ______  (measure, never estimate)
```

> **The stage estimates above are engineering estimates, not measurements.** NFR-01 is satisfied only by a recorded `make bench-ingest` run (§22). Do not put estimated numbers in the README, a slide, or a sentence spoken to a judge.

12.2 Engineering rules [DEC]

- **One pass, no row loops.** Any `for` over transactions is a bug. All transforms are vectorised Polars/DuckDB expressions.
- **Row conservation is asserted, not hoped for.** Every stage logs `rows_in → rows_out`; the pipeline aborts on any unexplained change (D1).
- **Profile before assuming** (P8). Stage [0] runs on first ingest; stages [2] and [4] read their enum domains and signal definitions from `config.py`, and the ingest report flags any value the profiler did not expect.
- **Category encoding before anything else.** Strings are the memory hog; convert to `int32` / `uint8` early.
- **Streaming guard.** If rows > 5 M or available RAM < 8 GB, switch to Polars `collect(streaming=True)` + chunked CSR build.
- **Idempotent + cached.** Re-ingesting a file whose SHA-256 matches the manifest skips to artifact load (cold start ≤ 3 s). Demo-day insurance — **never ingest live in front of a judge.**
- **Progress reporting** streamed to the UI via `GET /api/ingest/status`.
- **Timing table printed and saved** — this is the NFR-01 evidence.

12.3 Reference SQL — dual-sided aggregation in one pass [DEC]

Both directions are always computed (P7). Exact duplicates are collapsed for aggregates only (D6).

```
CREATE OR REPLACE TABLE acct_out AS
SELECT src_idx                                  AS a,
       COUNT(*)                                 AS out_txn_count,
       SUM(Amount)                              AS out_amount,
       COUNT(DISTINCT dst_idx)                  AS out_degree,
       COUNT(DISTINCT receiver_bank)            AS out_distinct_banks,
       AVG(Amount)                              AS out_avg_amount,
       MAX(Amount)                              AS out_max_amount,
       MIN(ts_epoch)                            AS out_first_ts,
       MAX(ts_epoch)                            AS out_last_ts,
       SUM(CASE WHEN terminal_signal_mask > 0 THEN 1 ELSE 0 END) AS out_signal_txns,
       SUM(CASE WHEN amount_round THEN 1 ELSE 0 END)             AS out_round_amt
FROM transactions
WHERE NOT is_exact_duplicate OR is_duplicate_primary      -- D6
GROUP BY src_idx;

-- symmetric acct_in on dst_idx (in_txn_count, in_amount, in_degree,
-- in_distinct_banks, in_avg_amount, in_max_amount, in_first_ts, in_last_ts,
-- in_signal_txns), then FULL OUTER JOIN into `features`.
-- FULL OUTER is required: receive-only and send-only accounts must both appear.
```

12.4 Duplicate report (stage 5b) [DEC — new in v2.1]

```
CREATE OR REPLACE TABLE duplicate_report AS
SELECT 'txn_id_repeat' AS kind, occ AS repeat_count, COUNT(*) AS n_ids,
       COUNT(*) * occ AS n_rows
FROM (SELECT Transaction_ID, COUNT(*) AS occ FROM transactions
      GROUP BY Transaction_ID HAVING COUNT(*) > 1)
GROUP BY occ
UNION ALL
SELECT 'exact_duplicate', 1, COUNT(*), COUNT(*) FROM transactions WHERE is_exact_duplicate;
```

Surfaced on the Ingest screen as: "1,997,748 distinct Transaction_IDs across 2,000,000 rows. N IDs appear more than once, affecting M rows. K rows are exact duplicates. All rows retained." **[DATA-PRELIM — N, M, K come from the profiler]**

12.5 Acceptance (NFR-01)

`make bench-ingest` prints total wall-clock, per-stage breakdown, peak RSS, rows/second, **row-conservation assertion**, duplicate statistics, and **PASS/FAIL vs the 60 s target**. Three consecutive runs must pass.

---

13. Graph Engine Specification

13.1 CSR representation [DEC] — resized in v2.1

Edges are aggregated per unique (src, dst) pair for traversal; individual source rows stay in DuckDB for evidence. On this dataset aggregation is near-identity (≈99.8% of rows are their own pair, §7.6).

```
# forward (money-out) adjacency, sorted by src then dst
out_indptr     : int32[N+1]    # N ≈ 24,873
out_indices    : int32[E]      # destination node index          E ≈ 2.0 M  [DATA-PRELIM]
out_weight     : float64[E]    # total ₹ moved on this pair
out_count      : int32[E]      # number of source rows on this pair
out_first_ts   : int64[E]      # earliest transaction on this pair
out_last_ts    : int64[E]      # latest transaction on this pair
out_signal     : uint8[E]      # OR of terminal_signal_mask over the pair   ← new in v2.1
# in_* mirror, required for upstream tracing and for dual-sided features (P7)
```

Memory (recomputed): ~32 B/edge × 2.0 M ≈ **64 MB forward + 64 MB reverse ≈ 128 MB [DATA-PRELIM]** — five times the v2.0 estimate, and still comfortably inside budget. Verify against `profile.json → unique_pairs` before trusting it.

Row-level evidence lookup: an edge does not store its source rows inline. `edge_rows(src, dst, t0, t1)` queries DuckDB on the `(src_idx, ts_epoch)` index and returns `source_row_id`s — keeping the CSR compact while preserving full provenance (D7).

13.2 Operations

Op	Signature	Complexity	Budget
Neighbours (out / in)	`neighbors(u, dir)`	O(deg)	µs
Forward trace	`trace(src, depth=4, …)`	O(visited·deg) pruned	≤ 300 ms
Upstream trace	`trace(dst, direction="up")`	same	≤ 300 ms
k-hop subgraph	`ego(u, radius=2, cap=800)`	pruned BFS	≤ 200 ms
Rings	union-find over the high-risk mask	O(N+E)	ingest-time
Cycles ≤ 4	DFS with depth cap on the high-risk subgraph	—	ingest-time

13.3 Hub guard [DEC] — recalibrated in v2.1

The average out-degree on this dataset is ≈80 **[DATA-PRELIM]**, so the v2.0 absolute cap of 2,000 is ~25× the mean and may never trigger. The guard becomes **distribution-relative**:

```
HUB_DEGREE_CAP = max(HUB_DEGREE_FLOOR,           # 500, absolute safety floor
                     profile.out_degree_p99 * HUB_P99_MULTIPLIER)   # default 3.0
```

Behaviour is unchanged: a hub may be **entered** (it can legitimately receive stolen funds) but is **not expanded through** unless inbound traced value ≥ `HUB_PASSTHROUGH_MIN`, and then only its top-K outgoing edges by value within the temporal window. Hubs render with a distinct style and an "⚠ aggregator — not expanded" note, are labelled `AGGREGATOR`, and are **excluded from freeze recommendations**.

Phase-1 task for M3: read `profile.json → out_degree p50/p90/p99/max` and set the multiplier so the guard marks a small, defensible set of accounts. A hub guard that never fires is as bad as one that fires on everything.

14. Detection Engine Specification

v2.1 restructures this section around principle P7: every account is characterised from BOTH sides of the ledger, always. A "collector" that only collects and a "distributor" that only distributes do not exist in this data model (§2.4).

14.1 Feature catalogue — dual-sided, ~45 features per account [DEC]

Computed for **every** account in a single ingest-time pass (§12.3). Receive-only and send-only accounts are produced by a FULL OUTER JOIN, with the absent side set to zero — never missing.

A · INCOMING profile

Feature	Definition
`in_txn_count`	number of credit transactions
`in_amount`	total value received
`in_degree`	distinct senders
`in_distinct_banks`	distinct sender banks (IFSC prefix)
`in_avg_amount`, `in_max_amount`	average / largest credit
`in_first_ts`, `in_last_ts`, `in_active_days`	inbound activity span
`in_burst_score`	max credits in any 10-min window ÷ mean rate
`in_temporal_concentration`	share of inbound value arriving in the busiest hour
`in_signal_ratio`	share of credits carrying a terminal signal

B · OUTGOING profile

Feature	Definition
`out_txn_count`	number of debit transactions
`out_amount`	total value sent
`out_degree`	distinct receivers
`out_distinct_banks`	distinct receiver banks
`out_avg_amount`, `out_max_amount`	average / largest debit
`out_first_ts`, `out_last_ts`, `out_active_days`	outbound activity span
`out_burst_score`	max debits in any 10-min window ÷ mean rate
`out_temporal_concentration`	share of outbound value leaving in the busiest hour
`split_consistency`	coefficient of variation of sibling outgoing amounts within a dispersion event
`downstream_count`	distinct receivers inside the dispersion window (the "3–7" test)

C · CROSS-FLOW (the relationship between the two sides — where mule behaviour lives)

Feature	Definition
`fan_in_ratio`	`in_degree / max(in_txn_count,1)` — many senders or one sender repeatedly?
`fan_out_ratio`	`out_degree / max(out_txn_count,1)`
`in_out_amount_ratio`	`out_amount / max(in_amount, ε)`
`in_out_txn_ratio`	`out_txn_count / max(in_txn_count, 1)`
`degree_balance`	`(in_degree − out_degree) / (in_degree + out_degree)` → +1 pure collector shape, −1 pure distributor shape, 0 balanced. **Used to decide the dominant role, not to exclude the other.**
`pass_through_ratio`	temporally matched — see §14.2, **not** `out_amount/in_amount`
`sink_ratio`	`1 − matched_out_value / in_amount` — how much received value stops here
`median_hold_time`, `p10_hold_time`	seconds between a credit and the matched onward debit
`rapid_dispersion_count`	matched dispersions completing inside `VELOCITY_WINDOW_SEC`
`counterparty_reuse`	share of transactions with repeat counterparties
`new_counterparty_rate`	distinct new counterparties per active day

D · TERMINAL indicators (evidence, never verdicts — P10)

Feature	Definition
`terminal_signal_mask_or`	bitwise OR of signals over the account's transactions
`terminal_signal_distinct`	**number of distinct signal types after the co-occurrence discount** (§7.4)
`terminal_sig_ratio`	share of the account's transactions carrying any signal
`foreign_ip_ratio`, `automation_ratio`, `crypto_narr_ratio`, `wallet_narr_ratio`	per-signal shares, retained for explanation text
`low_onward_transfer`	`sink_ratio ≥ SINK_RATIO_MIN` **and** `in_amount ≥ SINK_MIN_AMOUNT`
`distinct_ip_count`, `distinct_device_count`	client diversity

E · BEHAVIOURAL

`burst_score` (max of in/out) · `round_amount_ratio` · `amount_entropy` (Shannon over log-binned amounts) · `dormant_to_burst` (quiet ≥ D days then a burst) · `night_ratio` · `cycle_member` · `ring_id`, `ring_size` · `pagerank` (money-weighted, 20 iterations, vectorised)

> **No single feature classifies an account.** Each feeds a rule card with a bounded contribution and a reason string; the MRI caps any one component's influence (§15.2). High transaction volume in particular is **not** evidence of mule behaviour — on this dataset the average account already makes ≈80 transfers **[DATA-PRELIM]**.

14.2 Pass-through: temporally matched, not ratio-of-totals [DEC — rewritten in v2.1]

v2.0 computed `pass_through_ratio = out_amount_in_window / in_amount`. That is wrong for an account with many credits and many debits spread over 15 days: it conflates unrelated money.

Matched pass-through algorithm

```
For each account A:
  credits = debits = []  (sorted by timestamp)
  For each credit c (ascending ts):
      remaining = c.amount
      window    = [c.ts + MIN_HOLD_SEC , c.ts + PASSTHROUGH_MATCH_SEC]
      For each debit d in window with unmatched capacity, ascending ts:
          take = min(remaining, d.unmatched)
          record match (c, d, take)          # FIFO attribution
          remaining     -= take
          d.unmatched   -= take
          if remaining <= 0: break
      c.matched = c.amount - remaining

  pass_through_ratio   = Σ c.matched / Σ c.amount
  median_hold_time     = median over matches of (d.ts - c.ts), value-weighted
  rapid_dispersion_cnt = number of credits whose matched debits all fall
                         inside VELOCITY_WINDOW_SEC  (180–900 s)  [PS]
  downstream_count     = distinct receivers across a credit's matched debits
  sink_ratio           = 1 - pass_through_ratio
```

FIFO attribution is the declared convention **[DEC]** — stated in the PRD, in `docs/EVALUATION.md` and in the methodology note of generated documents, because a different forensic convention (LIFO, pro-rata) would produce different numbers and we must not pretend ours is the only one.

Edge cases — specified, each with a test (§25)

Case	Behaviour
Credit with no debit in window	`matched = 0`; contributes to `sink_ratio`, not to velocity
Debit exceeding the matched credit	Matched up to the credit; the surplus is the account's own funds and is excluded from pass-through
Many small credits → one large debit	Aggregation is handled naturally; all contributing credits are matched
One large credit → many small debits	The "3–7 split" case; `downstream_count` counts distinct receivers
Debit **before** any credit	Never matched — temporal causality (money cannot leave before it arrives)
Debit inside `MIN_HOLD_SEC` (<180 s)	Matched, flagged `instant_forward`; scored under R1 but reported separately — sub-3-minute forwarding is outside the PS window and may be ordinary activity
Account with credits only	`pass_through_ratio = 0`, `sink_ratio = 1` — a terminal candidate, pending other evidence
Account with debits only	Not a pass-through candidate; excluded from R1/R3
Exact-duplicate rows	Counted once in matching (D6), both visible in evidence
Amount conservation	`Σ matched ≤ Σ credits` and `Σ matched ≤ Σ debits` are asserted in tests

All thresholds (`MIN_HOLD_SEC`, `PASSTHROUGH_MATCH_SEC`, `VELOCITY_WINDOW_SEC`, `PASS_THROUGH_RATIO`) remain in `config.py`.

14.3 Rule cards — each returns `(fired, contribution, reason)` [DEC]

Rule	Condition (defaults in `config.py`)	Example reason text
R1 Pass-through [PS]	`pass_through_ratio ≥ 0.90` AND `median_hold_time ∈ [180, 900] s` AND `out_txn_count ≥ 2`	"Dispersed 94% of matched credits within a median of 6 m 12 s across 5 transfers."
R1b Near pass-through	`pass_through_ratio ≥ 0.80` AND `median_hold_time ≤ 3600 s`	"Dispersed 83% of matched credits within 42 minutes."
R2 Collector shape [PS]	`in_degree ≥ COLLECTOR_MIN_IN_DEGREE` AND `degree_balance > 0.2` AND `in_distinct_banks ≥ 2`	"Received from 11 distinct accounts across 4 banks, and forwarded onward."
R3 Distributor shape [PS]	`downstream_count ∈ [3,7]` AND `split_consistency ≤ 0.35` AND `pass_through_ratio ≥ 0.7`	"Split an incoming ₹X into 5 near-equal transfers within 8 minutes."
R3b Wide fan-out	`downstream_count > 7` AND `pass_through_ratio ≥ 0.7`	"Dispersed to 12 downstream accounts — wider than the expected 3–7 pattern."
R4 Terminal evidence [PS]	`terminal_signal_distinct ≥ 1` AND `sink_ratio ≥ 0.5` AND `in_amount ≥ SINK_MIN_AMOUNT`	"Received ₹X, retained 87%, and showed Linux_Script activity from 185.x.x.x with P2P/ crypto narration."
R5 Sink	`sink_ratio ≥ 0.85` AND `in_amount ≥ SINK_MIN_AMOUNT`	"Retains 92% of received value — funds stop here."
R6 Burst	`burst_score ≥ 5`	"17 transactions in a 10-minute window, 5× this account's baseline."
R7 Structuring	`round_amount_ratio ≥ 0.7` AND `out_txn_count ≥ 4`	"87% of transfers in round ₹-thousand amounts."
R8 Cycle	`cycle_member`	"Participates in a circular fund loop of length 3."
R9 Dormant-to-burst	`dormant_to_burst`	"No activity for 9 days, then 12 transactions in one hour."
R10 Night operations	`night_ratio ≥ 0.6` AND `in_txn_count ≥ 5`	"73% of activity between 00:00 and 05:00."

R4 wording is normative. The reason string must describe combined evidence. `"Linux_Script ⇒ mule"` is a prohibited formulation (P10); the test suite greps generated reasons for bare-signal phrasing.

14.4 Terminal signals are weighted evidence, not proof [DEC — new in v2.1]

```
✘ "Device_Type = Linux_Script, therefore this account is a mule."

✔ "Terminal evidence increased for this account because it received ₹X from an
   upstream account already in the victim trail, retained 87% of that value, and
   the associated transactions originated from 185.22.x.x using Linux_Script with
   P2P/crypto narration. Terminal confidence: 0.81."
```

Rules: (a) a terminal signal alone never exceeds `TERMINAL_SIGNAL_SOLO_CAP` points of MRI; (b) signals are combined into **one** composite component after the co-occurrence discount (§15.1 C5); (c) every signal-derived score carries a reason string naming the signal, the amount and the flow context; (d) the signal definitions (IP prefixes, device names, keyword lists) are configurable, and the UI shows which configured rule matched.

14.5 Suppression rules — false-positive control [DEC]

Suppressor	Effect
`in_amount < SUPPRESS_MIN_AMOUNT` AND `in_txn_count < 3`	Cap MRI at 35 — too little evidence to accuse
`is_hub` (§13.3)	Label `AGGREGATOR/MERCHANT`, cap MRI at 50, exclude from freeze recommendations
High volume with low pass-through — `txn_count` in the top decile AND `pass_through_ratio < 0.3`	−15 MRI. **Busy is not guilty**; on this dataset the average account already transacts ≈80 times. (new in v2.1)
Regular periodic credit from a single dominant sender with low fan-out (salary-like)	−15 MRI
Active across the full window with steady, low-variance activity	−10 MRI
`out_degree = 0` AND no terminal signal AND `in_amount` below threshold	Not promoted to terminal on absence of outgoing alone (§2.4)

15. Mule Risk Index (MRI) — exact formula

[DEC] — the PS mandates a 0–100 index but not its formula. **The v2.0 structure is deliberately preserved**: six explainable components, configurable weights, reason strings. v2.1 changes only what the dataset forces: the terminal component is de-duplicated, and a no-single-feature-dominates cap is made explicit.

15.1 Component scores (each 0–1, then weighted)

Let `σ(x; k, x₀) = 1 / (1 + e^(−k(x − x₀)))`.

#	Component	Weight	Definition
C1	**Pass-through**	**25**	`clip(pass_through_ratio / PASS_THROUGH_RATIO, 0, 1)` — using the matched ratio of §14.2
C2	**Velocity**	**20**	`1 − clip((median_hold_time − 180) / (3600 − 180), 0, 1)` ; 0 when `out_txn_count = 0`
C3	**Collector shape (fan-in)**	**15**	`σ(in_degree; k=0.45, x₀=6) × clip(in_distinct_banks/3, 0, 1)^0.5`
C4	**Distributor shape (fan-out)**	**15**	`σ(downstream_count; k=0.60, x₀=4) × (1 − clip(split_consistency/0.5, 0, 1))`
C5	**Terminal evidence**	**15**	**rewritten in v2.1 — see below**
C6	**Behavioural anomaly**	**10**	mean of { `burst_norm` , `round_amount_ratio` , `night_ratio` , `cycle_member` }

C5 with the co-occurrence discount [DEC — new in v2.1]

Because the four terminal signals appear to be one injected composite (§7.4), blending them as independent evidence would quadruple-count a single fact:

```
base    = max(terminal_sig_ratio, 0.6*foreign_ip_ratio + 0.6*automation_ratio
                                + 0.5*crypto_narr_ratio + 0.5*wallet_narr_ratio)
# discount for signals that co-occur rather than corroborate independently
k       = terminal_signal_distinct              # distinct signal TYPES present
indep   = 1 + TERMINAL_INDEP_BONUS * (k - 1) * (1 - observed_cooccurrence_rate)
C5      = clip(base * indep / (1 + TERMINAL_INDEP_BONUS * (k - 1)), 0, 1) * context
context = clip(sink_ratio / SINK_RATIO_MIN, 0.3, 1.0)   # flow context required (P10)
```

`observed_cooccurrence_rate` is read from `profile.json` (FR-A05). **If the profiler confirms perfect co-occurrence (rate = 1.0), C5 collapses to a single-signal score — which is the correct behaviour.** If the signals turn out to be independent, the bonus restores corroboration value. Either way the behaviour is data-driven, not guessed.

The `context` multiplier enforces P10: a terminal signal on an account that immediately forwards everything onward is weak evidence of cash-out; the same signal on an account where the money stops is strong.

15.2 Adjustments and the dominance cap

```
RAW = 25·C1 + 20·C2 + 15·C3 + 15·C4 + 15·C5 + 10·C6                ∈ [0, 100]

MRI = clip( RAW · network_mult + proximity_bonus − suppression_penalty , 0 , 100 )

network_mult        = 1.00 normally
                    = 1.15 if in a ring with mean MRI ≥ 60 and ring_size ≥ 4
proximity_bonus     = +8 if ≤2 hops downstream of the victim in the active case
                      +4 if 3–4 hops
suppression_penalty = §14.5
```

Dominance cap [DEC — new in v2.1, enforces P10]

```
If a single component supplies more than SINGLE_COMPONENT_CAP (default 40) points
of RAW, the excess is halved before the adjustments are applied.

⇒ No account reaches the CRITICAL band on one feature alone.
⇒ TERMINAL_SIGNAL_SOLO_CAP (default 45) bounds an account whose ONLY positive
  evidence is C5 — a foreign IP with no supporting flow behaviour cannot, by itself,
  produce a freeze recommendation.
```

A unit test asserts both caps against synthetic single-signal accounts (§25).

15.3 Bands [DEC]

Score	Band	Colour	Operational meaning
0–29	**LOW**	slate	Likely legitimate; no action
30–59	**MODERATE**	amber	Monitor; appears in the trail, not in the freeze list
60–79	**HIGH**	orange	Suspected mule; included in the case diary
80–100	**CRITICAL**	red	Recommended for immediate freeze

Freeze recommendation: `MRI ≥ FREEZE_MRI_MIN (80)` **OR** (`MRI ≥ 60` **AND** currently holds traced victim funds > `FREEZE_HOLDING_MIN`) **AND NOT** `is_aggregator`. **Recalibrated in v2.1:** `FREEZE_HOLDING_MIN` drops from ₹10,000 to **₹2,000**, because the median transaction is ₹899.35 **[DATA-PRELIM]** and a ₹10,000 floor would exclude most genuinely traced holdings on this dataset.

15.4 Calibration protocol — explicitly heuristic [DEC]

> **GROUND-TRUTH LABEL AVAILABILITY: UNKNOWN (§29 Q1).** No labels are in our possession at the time of writing. Until a mentor confirms otherwise, **all threshold tuning in this project is heuristic self-validation, not supervised optimisation.** Every reported metric must state which regime produced it. Do not describe a self-validated number as "accuracy on the ground truth".

Regime	Method	How results must be labelled
Labels supplied	PR-curve sweep τ = 40…90, pick argmax F1 and a precision-constrained operating point	"Supervised evaluation on supplied labels"
No labels (current)	(a) synthetic mule injector on a copy of the real file; (b) hand-validated pattern cases; (c) unsupervised sanity checks — score distribution, rank stability, top-N manual review; (d) blind victim-trace tests	"**Heuristic self-validation** — no supplied ground truth"

Tune on injection seed 1, report on seed 2. Record the full sweep table, the confusion matrix, and the top-10 false positives with their reason lists.

15.5 Explainability contract [DEC]


```
{
  "account": "ILLUSTRATIVE-000000000000",
  "mri": 87,
  "band": "CRITICAL",
  "case_relative_role": "COLLECTOR",
  "role_note": "Role is relative to the victim case under investigation.",
  "layer": 1,
  "confidence": 0.86,
  "flow": {"in_txn": 14, "in_amount": 250000.0, "in_degree": 11,
           "out_txn": 5, "out_amount": 247000.0, "out_degree": 5},
  "components": {"pass_through": 25.0, "velocity": 18.4, "collector_shape": 6.2,
                 "distributor_shape": 14.1, "terminal": 7.5, "behavioural": 6.8},
  "caps_applied": ["none"],
  "multipliers": {"network_mult": 1.15, "proximity_bonus": 8, "suppression": 0},
  "reasons": [
    "Dispersed 98.8% of matched credits within a median of 4 m 10 s across 5 transfers.",
    "Received from 11 distinct accounts across 4 banks, and forwarded onward.",
    "Split the incoming value into 5 near-equal transfers."
  ],
  "evidence_source_row_ids": [1048221, 1048307, 1048412]
}
```


(Values above are illustrative placeholders, not dataset observations.) A score with no reasons is a bug; a reason with no `source_row_id` is a bug.

---

16. Layer Classification Algorithm (L1/L2/L3)

Rewritten in v2.1. Layers are decided by **ten inputs**, never by hop depth alone, never by in-degree alone, never by out-degree alone. Every mule role both receives and sends (§2.4, P7); the layer expresses where the account sits in this victim's trail and which behaviour dominates.

16.1 Inputs to the classifier [DEC]

#	Input	Source
1	Victim-relative graph position	trace result
2	Hop depth from the victim	trace result
3	Incoming behaviour (`in_degree` , `in_distinct_banks` , `in_amount` , inbound burst)	§14.1 A
4	Outgoing behaviour (`out_degree` , `out_amount` , outbound burst)	§14.1 B
5	Pass-through behaviour (matched ratio, `sink_ratio`)	§14.2
6	Fan-in strength	§14.1 C
7	Fan-out strength	§14.1 C
8	`downstream_count` within the dispersion window (the 3–7 test)	§14.1 B
9	Terminal evidence (composite, with flow context)	§14.1 D, §14.4
10	Temporal relationship to the upstream transaction that funded it	trace result

16.2 Algorithm

```
INPUT  victim_idx, trace T (per node: hop_depth, traced_in, traced_out, t_funded)
       features F (dual-sided, per account)

STEP 1 — POSITION
  depth(n)        = hop_depth in the victim-relative trail
  funded_by(n)    = the upstream node that delivered traced value
  t_funded(n)     = timestamp of that delivery
  responds(n)     = TRUE if n has outgoing activity AFTER t_funded(n)
                    within RESPONSE_WINDOW_SEC        (temporal relationship, input 10)

STEP 2 — ROLE EVIDENCE  (computed independently of depth; all may be non-zero)
  collector_score(n)  = w1·norm(in_degree) + w2·norm(in_distinct_banks)
                      + w3·max(0, degree_balance) + w4·pass_through_ratio
  distributor_score(n)= w5·in_range(downstream_count, 3, 7)
                      + w6·(1 − split_consistency) + w7·pass_through_ratio
                      + w8·max(0, −degree_balance) + w9·received_from_upstream_mule
  terminal_score(n)   = w10·C5_terminal_evidence + w11·sink_ratio
                      + w12·(1 − responds(n))          # weak input, NOT decisive
  # Each score ∈ [0,1]; weights in config.py; every term contributes a reason string.

STEP 3 — ASSIGNMENT  (position gates, behaviour decides)
  if depth(n) == 0:                                    LAYER 0  VICTIM
  elif terminal_score(n)  ≥ TERMINAL_MIN   and
       terminal_score(n)  ≥ distributor_score(n):      LAYER 3  TERMINAL
  elif depth(n) == 1:
        LAYER 1 COLLECTOR
        ↳ but if collector_score < COLLECTOR_MIN and distributor_score is dominant,
          emit role = DISTRIBUTOR-AT-L1 with reduced confidence and say so.
  elif depth(n) == 2:
        LAYER 2, role = DISTRIBUTOR if distributor_score ≥ DISTRIBUTOR_MIN
                        else INTERMEDIARY
  elif depth(n) >= 3:
        LAYER 3 if terminal evidence OR no onward flow of traced value
        else    LAYER 3 (deep chain, hop k) — reported with its true hop depth

STEP 4 — CONFIDENCE
  confidence = 0.4·position_agreement        # does depth agree with the chosen role?
             + 0.4·role_margin               # winning score minus runner-up
             + 0.2·evidence_volume           # how many independent rules fired
  Badge: ●●● ≥0.75 high · ●●○ 0.45–0.75 medium · ●○○ <0.45 low

STEP 5 — PRESENTATION
  The UI always presents the three PS layers. Hops 3–4 that are not terminal are shown
  as "L3 (deep chain, hop k)" with the true depth preserved in the payload.
```

16.3 Rules that are explicitly forbidden

✘ Never	Why
`layer = hop_depth`	Real cash-out can occur at hop 2; a rigid mapping mislabels short chains
`in_degree high ⇒ L1`	A distributor also receives; a merchant has high in-degree and is not a mule
`out_degree high ⇒ L2`	A collector also sends; payroll accounts have high out-degree
`out_degree == 0 ⇒ L3`	Absence of outgoing rows is weak evidence, contributing at most `w12` . A dormant victim account has no outgoing activity either.
`terminal signal ⇒ L3`	A signal without flow context is not cash-out evidence (P10, §14.4)

16.4 Output contract

```
{
  "layer": 2,
  "role": "DISTRIBUTOR",
  "role_is_case_relative": true,
  "confidence": 0.71,
  "hop_depth": 2,
  "scores": {"collector": 0.34, "distributor": 0.78, "terminal": 0.12},
  "reasons": [
    "Received traced value from a Layer-1 account 3 m 41 s earlier.",
    "Forwarded to 5 downstream accounts in near-equal amounts.",
    "Dispersed 91% of matched credits within a median of 7 m 02 s."
  ]
}
```

Every account also carries a **global modal role** across all analysed cases (§2.5), so the system can answer the evaluation either way (§29 Q10).

16.5 Worked classification cases (illustrative, used as fixtures in §25 D)

Case	in	out	downstream	pass-through	terminal	Expected
Receives only, small amount, no signal	3	0	0	0	none	Not auto-L3; MODERATE at most, reason states absence of outgoing is weak evidence
Receives from 12, forwards to 4 rapidly, at hop 1	12	4	4	0.95	none	**L1 COLLECTOR**, high confidence
Receives from 1 upstream, splits to 5, at hop 2	1	5	5	0.93	none	**L2 DISTRIBUTOR**, high confidence
High in **and** high out (in 15 / out 12), hop 1	15	12	12	0.88	none	**L1**, role margin reported; distributor score surfaced in the inspector
Exactly 3 downstream	1	3	3	0.9	none	L2, inside the PS 3–7 band
Exactly 7 downstream	1	7	7	0.9	none	L2, inside the band
12 downstream	1	12	12	0.9	none	L2 with **R3b wide fan-out** reason; still L2
Receives ₹X, retains 90%, foreign IP + Linux_Script	4	1	1	0.1	strong	**L3 TERMINAL**, high confidence
Receives ₹X, retains 90%, **no** signal	4	0	0	0	none	**L3 TERMINAL** on sink evidence, medium confidence, reason names the absence
Same account in two different victim traces	—	—	—	—	—	May carry **different** layers; both stored, global modal role unchanged

17. Money-Trail Tracing Algorithm (4-hop, value-weighted)

This is the 40%-weighted feature. v2.1 keeps the v2.0 algorithm unchanged in structure and recalibrates three constants that the real dataset proves wrong.

17.1 Requirements

Given a victim account (and optionally a complaint timestamp and disputed amount), return the downstream flow of that money — not every transaction every downstream account ever made.

17.2 Algorithm — Temporally-Ordered Value-Attributed BFS [DEC]

```
def trace(victim, t0=None, depth=4, min_value=None, window=TRACE_WINDOW_SEC, topk=15):
    """
    t0        : complaint time (default = victim's first outgoing transaction)
    min_value : absolute floor; default max(TRACE_MIN_VALUE_ABS,
                                            TRACE_MIN_VALUE_REL * initial_amount)
    window    : max seconds after the inbound credit that an outbound may occur
    topk      : max outgoing edges expanded per node, by traced value
    """
    frontier = [(victim, initial_amount, t0, 0)]
    traced_in, edges = defaultdict(float), []

    while frontier:
        u, value_at_u, t_in, hop = frontier.pop()
        if hop >= depth: continue
        cands = txns_from(u, after=t_in, before=t_in + window)      # causality
        if not cands: continue
        total_out = sum(c.amount for c in cands)
        for c in top_by_amount(cands, topk):
            share  = c.amount / max(total_out, c.amount)
            traced = min(value_at_u * share, c.amount)              # attribution
            if traced < min_value: continue                         # pruning
            if is_hub(c.dst) and traced < HUB_PASSTHROUGH_MIN: continue
            edges.append(Edge(u, c.dst, c.txn_id, c.source_row_id,
                              c.amount, traced, c.ts, hop+1))
            traced_in[c.dst] += traced
            if c.dst not in seen_at_shallower_hop:
                frontier.append((c.dst, traced, c.ts, hop+1))
            else:
                mark_cycle(u, c.dst)
    return assemble(edges, traced_in)
```

Properties (unchanged)

Temporal causality · **proportional attribution** · **value pruning** · **hub guard** · **loop safety** · **determinism** (byte-identical output for identical input — required for golden-file tests and a reproducible demo).

Recalibrated constants [DEC — new in v2.1]

Constant	v2.0		Evidence
`TRACE_WINDOW_SEC`	72 h	**6 h** (presets 1 h / 6 h / 24 h / 72 h in the UI)	§7.5 — at 72 h the branching factor is ≈16 and hop 4 reaches ≈66,885 > the 24,873-account population: the trace returns the whole graph
`TRACE_MIN_VALUE`	₹1,000 flat	**`max(₹100, 0.2% × initial_amount)`**	§7.1 — the median transaction is ₹899.35, so a flat ₹1,000 floor prunes more than half of all single transfers and truncates genuine trails at hop 1–2

Constant	v2.0		Evidence
`TRACE_TOPK_PER_NODE`	25	**15**	With a 6 h window the expected branching is ≈1.3, so 15 is generous; it bounds the worst case on bursty accounts
`HUB_DEGREE_CAP`	2,000 absolute	**`max(500, p99_out_degree × 3)`**	§13.3 — mean out-degree is ≈80, so an absolute 2,000 may never fire

> Every one of these is in `config.py` and adjustable at request time via query parameters, so an investigator (or a judge) can widen the window and watch the result grow. **Show this during the demo** — it demonstrates that the pruning is a deliberate, measured choice.

17.3 Returned structure (all values illustrative)

```
{
  "victim": "ILLUSTRATIVE-000000000000",
  "query_ms": 0,
  "params": {"depth": 4, "window_sec": 21600, "min_value": 100, "topk": 15},
  "total_siphoned": 0.0,
  "total_traced": 0.0,
  "attribution_coverage": 0.0,
  "nodes": [
    {"account": "ILLUSTRATIVE-000000000001", "hop": 1, "layer": 1, "role": "COLLECTOR",
     "role_is_case_relative": true, "confidence": 0.86, "mri": 0,
     "flow": {"in_txn": 0, "in_amount": 0, "in_degree": 0,
              "out_txn": 0, "out_amount": 0, "out_degree": 0},
     "traced_in": 0, "traced_out": 0, "held": 0, "reasons": []}
  ],
  "edges": [
    {"from": "ILLUSTRATIVE-000000000000", "to": "ILLUSTRATIVE-000000000001",
     "txn_id": "<source value>", "source_row_id": 0, "evidence_row_id": "<fp>:0",
     "txn_id_is_duplicated": false,
     "amount": 0.0, "traced_value": 0.0, "ts": "2026-09-__T__:__:__",
     "mode": "UPI", "hop": 1, "narration": "", "ip": "", "device": ""}
  ],
  "layers": {"L1": [], "L2": [], "L3": []},
  "currently_holding": [],
  "recommended_freeze": [],
  "cycles": [],
  "truncated": false,
  "truncation_reason": null,
  "dataset_fingerprint": "sha256:<fill at runtime>"
}
```

Every edge carries `source_row_id` so that any claim in any document resolves to one exact CSV row, even when `Transaction_ID` is duplicated (§11.3 D8).

17.4 Latency budget (p95 target ≤ 2,000 ms [PS])

Stage	Budgeted	Measured
Account lookup + validation	5 ms	___
CSR traversal (4 hops, pruned, 6 h window)	60 ms	___
Evidence row fetch from DuckDB (indexed)	250 ms	___
Layer classification + risk join	40 ms	___
Response assembly + JSON serialisation	120 ms	___
Backend total	**~475 ms**	___
Frontend parse + Cytoscape layout	~600 ms	___
End-to-end perceived	**~1.1 s**	___

Budgets are design targets, not results. The "Measured" column is filled by `make bench-trace` (§22) and nowhere else.

17.5 Upstream trace [DEC]

`/api/trace/{acct}?direction=up` answers "who funded this account?". **Defensive design:** if a blind-test ID has no plausible downstream flow, the UI detects it and offers the upstream view instead of an empty graph. On this dataset — where every account averages ≈80 inbound and ≈80 outbound transactions **[DATA-PRELIM]** — an empty downstream result almost certainly means the ID is a terminal or an ordinary account, not that the system failed. This fallback protects the 40% criterion.

---

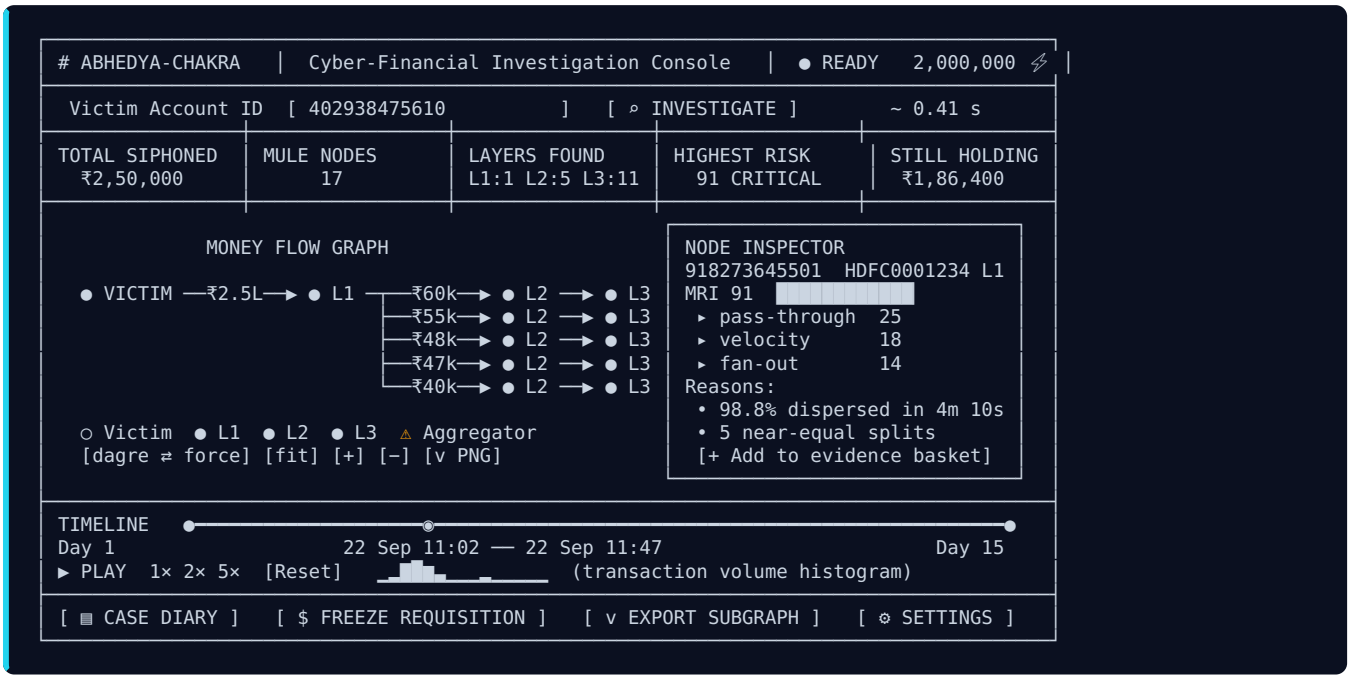
18. Frontend / UX Specification

18.1 Visual language [DEC]

Dark "operations console": background `#0b1020`, panels `#131a2e`, text `#e2e8f0`, accent cyan `#22d3ee`, layer colours per §15.3. Monospace (`JetBrains Mono`) for account numbers, amounts and transaction IDs — reads as forensic, and aligns digits. Indian number formatting throughout (`₹2,50,000`), 24-hour timestamps with the date.

18.2 Main dashboard

> Wireframe. Every value shown is an illustrative placeholder, not a dataset observation.



18.3 Screens

#	Screen	Purpose	Priority
S1	**Ingest / System Status**	Dataset load, progress bar, manifest + benchmark readout	MUST
S2	**Investigation Console** (above)	The demo screen	MUST
S3	**Account Explorer**	Search any account, transaction table, counterparties, mini-graph	MUST
S4	**Ring Board**	Top mule rings by value, sortable table → click to open graph	SHOULD
S5	**Document Preview**	Rendered case diary / requisition with Download PDF-DOCX	MUST
S6	**Evaluation Panel**	Precision/recall/F1, confusion matrix, benchmark table (judge-facing)	SHOULD

18.4 Graph interaction spec

- **Layout:** `dagre` left-to-right (rank = layer) by default — this reproduces the PS diagram and reads instantly. Toggle to `fcose` for ring topology.

- **Node encoding:** size ∝ log(traced value); fill = layer colour; border thickness ∝ MRI; icon badge for terminal signals (🌐 foreign IP, 🤖 automation, BTC crypto).
- **Edge encoding:** width ∝ log(amount); label = `₹amount` on hover; colour = red if within the 3–15 min velocity window; animated dash on Play.
- **Interactions:** click = inspect; double-click = isolate ego-network; right-click = context menu (trace from here, trace upstream, add to basket, copy account no.); hover = tooltip with txn details; box-select = multi-add to basket.
- **Performance:** `hideEdgesOnViewport: true`, `textureOnViewport: true`, `motionBlur: false`, batched `cy.batch()` updates, memoised stylesheet, layout run once per result (not per filter — filtering toggles `display:none` only).

18.5 Empty, loading & error states (judged harshly if missing)

- **Loading:** skeleton KPI cards + "Tracing 4 hops…" with a live ms counter.
- **Account not found:** "No transactions found for that account. Did you mean <nearest match>?" (Levenshtein ≤ 2 suggestion from the account dictionary).
- **Account found but no downstream flow:** "This account has no outgoing transfers in the window. It may itself be a terminal node — **[View upstream sources]**."
- **Truncated graph:** amber banner "Showing top 800 of 2,431 nodes by traced value. [Expand all] [Export full subgraph]".
- **Backend down:** full-screen reconnect card with the uvicorn command to restart. (3 a.m. insurance.)

18.6 Accessibility & demo hygiene

Keyboard: `/` focuses search, `Enter` investigates, `Esc` closes panels, `D` opens diary, `F` opens freeze notice. Colour-blind-safe palette (layer colours differ in lightness, plus shape badges). Minimum 14 px font; the projector at a hackathon is not a Retina display — **test the demo at 1366×768**.

---

19. API Specification

Base: `http://localhost:8000/api` · JSON · OpenAPI auto-published at `/docs`.

19.1 Endpoints

Method	Path	Purpose	p95 target
`GET`	`/health`	Liveness + state summary	10 ms
`POST`	`/ingest`	Start ingest `{path, force}` → `{job_id}`	async
`GET`	`/ingest/status`	Stage, pct, timings, manifest	10 ms
`GET`	**`/profile`**	**FR-A05 dataset profile (`profile.json`)** (new in v2.1)	50 ms
`GET`	**`/profile/duplicates`**	**ID-repeat histogram, affected rows, exact duplicates** (new in v2.1)	50 ms
`GET`	`/search/account/{id}`	Account profile, **both-sided flow**, MRI, reasons	150 ms
`GET`	`/search/suggest?q=`	Prefix/fuzzy account suggestions	50 ms
`GET`	`/account/{id}/transactions`	Paginated rows (`?dir=in|out|all&from=&to=&limit=&offset=`), each with `source_row_id`	300 ms
`GET`	**`/transaction/{txn_id}`**	**Returns a LIST — `Transaction_ID` is not unique** (new in v2.1)	50 ms
`GET`	**`/row/{source_row_id}`**	**Canonical single-row lookup** (new in v2.1)	20 ms
`POST`	`/investigate`	**Primary.** `{account_id, t0?, depth=4, window_sec?, min_value?, topk?}`	**2 s**
`GET`	`/trace/{id}`	GET convenience (`?direction=down|up&window_sec=`)	2 s
`GET`	`/risk/{id}`	MRI, components, caps applied, reasons	50 ms
`GET`	`/graph/{id}`	Cytoscape elements for a trace	1 s
`GET`	`/graph/{id}/subgraph`	Ego network (`?radius=2&cap=800`)	500 ms
`GET`	`/timeline/{id}`	Per-minute buckets for the slider histogram	200 ms
`GET`	`/rings`	Top mule rings `?limit=20&min_mri=60`	200 ms
`GET`	`/stats/overview`	Dataset KPIs for the status screen	50 ms
`POST`	`/evidence/build`	`{victim, selected_accounts[]}` → validated EvidenceObject	500 ms
`POST`	`/case-diary`	`{evidence_id, use_llm}` → `{doc_id, markdown, html, validation}`	10 s
`POST`	`/freeze-requisition`	`{evidence_id, bank_filter?}` → one requisition per beneficiary bank	10 s
`GET`	`/documents/{doc_id}.pdf|.docx|.md`	Download	2 s
`GET`	`/export/subgraph`	`?account=&format=csv|json|graphml` — CSV includes `source_row_id`	2 s
`GET`	`/eval/metrics`	Detection metrics **with a `regime` field: `supervised` | `heuristic_self_validation`**	100 ms

Method	Path	Purpose	p95 target
`GET`	`/bench`	Last recorded benchmark results (empty until measured)	10 ms

19.2 Conventions [DEC]

- Account numbers are **always strings** (leading zeros are real).
- Amounts are numbers in rupees (2 dp); the UI formats as `₹X,XX,XXX`.
- Timestamps are ISO-8601 local-naive, exactly as supplied — **never converted**.
- **Every transaction object carries `source_row_id`, `evidence_row_id` and `txn_id_is_duplicated`** (new in v2.1).
- Every response carries `query_ms` and `dataset_fingerprint`.
- Pagination: `limit` (default 100, max 1000) + `offset`, with `total` in the body.
- Any endpoint that accepts a `Transaction_ID` returns a **collection**, never a single object.

19.3 Error envelope

```
{ "error": { "code": "ACCOUNT_NOT_FOUND",
             "message": "No transactions found for account <value>.",
             "suggestions": ["<nearest match>"],
             "request_id": "req_0000" } }
```

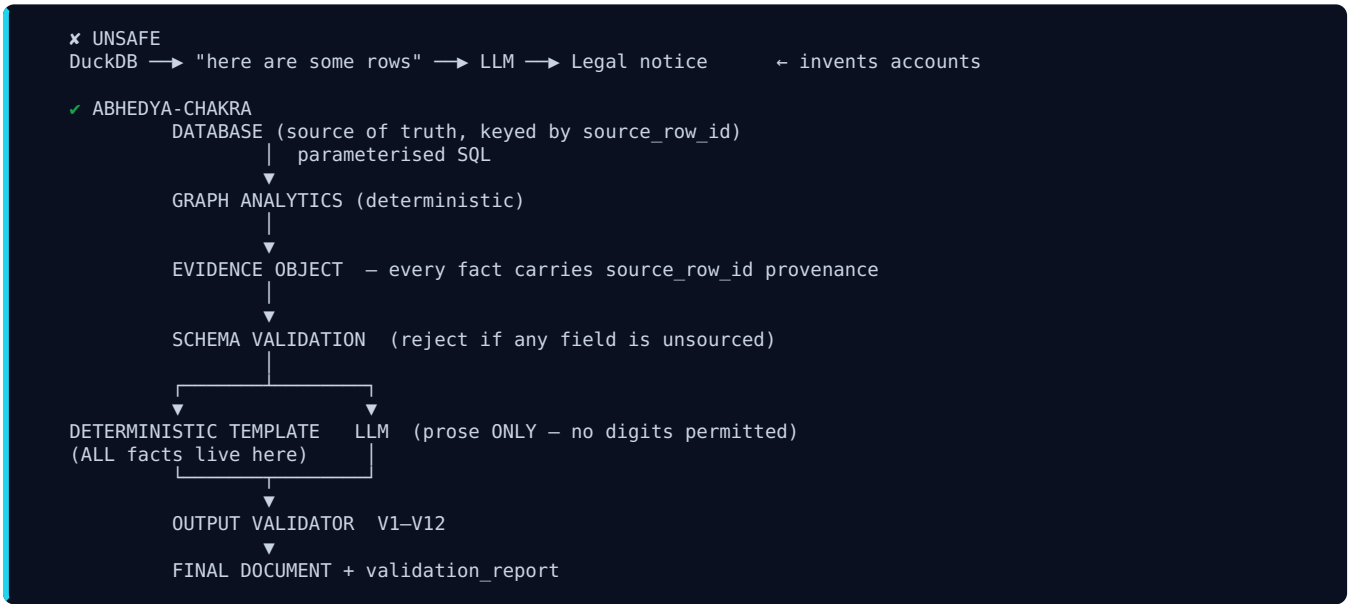
Codes: `ACCOUNT_NOT_FOUND`, `DATASET_NOT_LOADED`, `PROFILE_MISMATCH` (new — profiler contradicted §7.2), `INVALID_ACCOUNT_FORMAT`, `AMBIGUOUS_TRANSACTION_ID` (new — ID resolves to multiple rows where exactly one was required), `TRACE_TIMEOUT`, `EVIDENCE_VALIDATION_FAILED`, `LLM_UNAVAILABLE` (non-fatal), `INTERNAL`.

---

20. Evidence Object & Anti-Hallucination Architecture

An explicit PS requirement (FR-D03) and the most defensible part of the design. v2.1 adds row-level provenance so that every claim resolves to one exact CSV row even when `Transaction_ID` is duplicated.

20.1 Pipeline



20.2 The Evidence Object [DEC]

```
class TransactionEvidence(BaseModel):
    source_row_id: int                # PRIMARY provenance anchor  (new in v2.1)
    evidence_row_id: str              # "{fingerprint_short}:{source_row_id}"
    txn_id: str                       # source-provided, NOT assumed unique
    txn_id_is_duplicated: bool        # true → disambiguation is mandatory (D8)
    txn_id_occurrence: int            # 1-based index among rows sharing the ID
    is_exact_duplicate: bool
    sender: str; receiver: str
    sender_ifsc: str; receiver_ifsc: str
    amount: Decimal; timestamp: datetime
    mode: str; narration: str; ip: str | None; device: str | None

class AccountEvidence(BaseModel):
    account_no: str; ifsc: str; bank_code: str; bank_name: str | None
    layer: Literal[0,1,2,3]; role: str
    role_is_case_relative: bool = True          # always True (§2.5)
    confidence: float; mri: int
    flow_in: FlowSummary                        # count, amount, distinct senders  (new)
    flow_out: FlowSummary                       # count, amount, distinct receivers (new)
    traced_in: Decimal; traced_out: Decimal; currently_held: Decimal
    first_seen: datetime; last_seen: datetime
    reasons: list[str]                          # from rule cards only
    supporting_row_ids: list[int]               # source_row_id, not txn_id  (changed)

class EvidenceObject(BaseModel):
    evidence_id: str; case_id: str; generated_at: datetime
    dataset_fingerprint: str                    # SHA-256 of the source CSV
    dataset_profile_ref: str                    # hash of profile.json          (new)
    victim: AccountEvidence
    total_siphoned: Decimal; total_traced: Decimal; attribution_coverage: float
    attribution_method: str = "FIFO proportional"   # declared convention       (new)
    layer1: list[AccountEvidence]
    layer2: list[AccountEvidence]
    layer3: list[AccountEvidence]
    chronology: list[TransactionEvidence]       # sorted by timestamp
    currently_holding: list[AccountEvidence]
    recommended_freeze: list[AccountEvidence]
    beneficiary_banks: list[BankBlock]
    duplicate_notes: list[str]                  # cited IDs that are duplicated (new)
    config_snapshot: dict
    trace_params: dict                          # window, min_value, topk, depth (new)
```

20.3 Validator rules

#	Rule	On failure
V1	Every 12-digit token in the output ∈ accounts in the EvidenceObject	Reject
V2	Every transaction identifier in the output ∈ chronology	Reject
V3	Every ₹ amount ∈ evidence amounts ∪ computed totals (±₹1)	Reject
V4	Every IFSC ∈ evidence IFSCs and matches the regex	Reject
V5	Every date/time ∈ `[dataset_start, dataset_end]` and ∈ evidence timestamps	Reject
V6	Totals reconcile: Σ(L1 traced inflow) ≈ total_siphoned; Σ(held) ≤ total_traced	Reject
V7	No speculative language from the LLM (`probably`, `likely`, `it appears`, …)	Strip & warn
V8	**LLM output contains zero digits**	Regenerate
V9	Chronology is non-decreasing in timestamp	Sort & warn
V10	Document length within bounds; all required sections present	Warn
V11	**Every cited transaction resolves to exactly one `source_row_id`; if its `Transaction_ID` is duplicated, the citation carries date-time + amount + counterparty** (new in v2.1)	Reject
V12	**Every `source_row_id` cited exists in the fact table and its field values match the evidence record** (re-read from DuckDB, not from cache) (new in v2.1)	Reject

Validation report, shown in the UI and printed in the document footer:
`✔ n/n facts verified against source dataset · 0 unverified entities · m citations resolved to exact source rows · fingerprint <sha256>`

20.4 LLM usage policy [DEC]

- Local Ollama (`llama3.1:8b-instruct` or similar), temperature 0.2, ≤400 tokens.
- The prompt contains **only** the EvidenceObject summary — never raw dataset rows (SR-01, SR-03).
- The model writes exactly two things: a ~120-word "Summary of Investigation" and a ~60-word "Grounds for Freeze".
- System prompt: "You are drafting a factual police document. You may NOT state any number, account, amount, date or identifier. Refer to entities only as 'the victim account', 'the Layer-1 collector account', etc. Numeric facts are inserted by the system. Do not speculate. Do not assert that any technical indicator proves fraud."
- **`use_llm=false` produces a fully complete, legally formatted document.** Test the no-LLM path first; demo it by default.

---

21. Document Generation (Case Diary & S.94 BNSS Freeze Requisition)

21.1 Legal framing [verified]

Section **91 CrPC, 1973** (summons to produce document or other thing) is now **Section 94 of the Bharatiya Nagarik Suraksha Sanhita, 2023**, in force since 1 July 2024. Documents cite **"Section 94 BNSS, 2023 (corresponding to Section 91 CrPC, 1973)"**. Seizure/freeze powers reference **Section 106 BNSS (erstwhile Section 102 CrPC)** and the NCRP/Helpline-1930 workflow. A disclaimer states the document is a computer-generated draft for the Investigating Officer's verification and signature. **[DEC]**

> ### ⚠ All specimen values in §21.2 and §21.3 are ILLUSTRATIVE PLACEHOLDERS
>
> They are **template layout examples**, not observations from `VoidHacks8_MuleAccount_2M_Transactions.csv`. Account numbers, IFSC codes, transaction identifiers and amounts shown here are invented for formatting purposes and are marked as such. **No synthetic example may ever be presented as dataset evidence** (P1, §14.4). Dates are written inside the observed dataset window (2026-09-15 → 2026-09-29 **[DATA-PRELIM]**) so that the layout matches real output.

21.2 Case Diary — required content [PS]

```
            CASE DIARY — CYBER FINANCIAL FRAUD INVESTIGATION
          (Generated by ABHEDYA-CHAKRA Forensic Analysis Platform)
          *** ILLUSTRATIVE LAYOUT — PLACEHOLDER VALUES ***

Case ID        : <case id>                  Date: <generation datetime>
Investigating  : _____________________      P.S.: Cyber Cell, Indore
Dataset Ref    : sha256:<fingerprint>  (2,000,000 rows, 15-09 to 29-09)
Method         : FIFO proportional value attribution; window 6 h; 4 hops

1. COMPLAINT PARTICULARS
   Victim account       : <12-digit account>  (<IFSC>)
   Total amount siphoned: ₹<amount>
   Period of offence    : <first debit>  to  <last traced credit>

2. SUMMARY OF INVESTIGATION          ← the only LLM-written prose

3. CHRONOLOGY OF FUND MOVEMENT

   | #  | Date & Time        | From     | To       | Amount    | Mode | Row ID |
   |----|--------------------|----------|----------|-----------|------|--------|
   | 1  | <dd-mm-yy hh:mm>   | <acct>   | <acct>   | ₹<amt>    | UPI  | <srid> |
   | 2  | <dd-mm-yy hh:mm>   | <acct>   | <acct>   | ₹<amt>    | IMPS | <srid> |

   Each row cites Transaction_ID and source_row_id. Where a
   Transaction_ID occurs on more than one source row, the citation also
   carries date-time, amount and counterparty.

4. LAYERED MULE NETWORK IDENTIFIED
   (Roles are relative to this victim's money trail.)
   LAYER 1 — COLLECTOR ROLE
     <account> (<IFSC>) · MRI <score>/100 · confidence <c>
       in : <n> credits from <d> distinct senders across <b> banks
       out: <n> debits to <d> accounts
       — dispersed <p>% of matched credits within <t>
   LAYER 2 — DISTRIBUTOR ROLE  (<n> accounts, ₹<total>)
   LAYER 3 — TERMINAL / CASH-OUT  (<n> accounts, ₹<total>)
     <account> (<IFSC>) · MRI <score> · terminal evidence:
       received ₹<amt>, retained <p>%, activity from <ip> using
       <device> with <narration> narration

5. PRESENT STATUS OF FUNDS
   Accounts presently holding traced funds: ₹<total> across <n> accounts

6. ACCOUNTS RECOMMENDED FOR IMMEDIATE FREEZING
   Ranked by traced value held and MRI; aggregator accounts excluded.

7. TECHNICAL INDICATORS OBSERVED
   Stated as evidence contributing to assessment — not as proof of
   fraud in itself.

8. ANNEXURES
   A — Full transaction ledger (CSV, with source_row_id)
   B — Network graph (PNG)   C — Risk assessment sheet
   D — Validation report     E — Methodology & limitations note

✔ n/n facts verified against source dataset · 0 unverified entities
Computer-generated draft — to be verified and signed by the I.O.
```

Annexure E (new in v2.1) states, in plain language: the attribution convention (FIFO proportional), the traversal window and hop limit used, the fact that layer labels are relative to this victim's trail, and that technical indicators are contributing evidence rather than proof. This is what makes the document survive a competent cross-examination.

21.3 Freeze Requisition — required content [PS]

One requisition **per beneficiary bank**. (Layout below is illustrative.)

```
                    OFFICE OF THE COMMISSIONER OF POLICE
                       CYBER CRIME BRANCH, INDORE (M.P.)
              *** ILLUSTRATIVE LAYOUT — PLACEHOLDER VALUES ***

No. CYB/ABH/<year>/<serial>                              Date: <dd.mm.yyyy>

To,  The Nodal Officer, <Bank Name>

Sub:  Requisition for IMMEDIATE DEBIT FREEZE / LIEN and production of
      documents under Section 94 of the Bharatiya Nagarik Suraksha Sanhita,
      2023 (corresponding to Section 91 CrPC, 1973) read with Section 106
      BNSS, 2023 — in Crime No. ____/<year> u/s 318(4) BNS, 2023 r/w
      Section 66D, Information Technology Act, 2000.

Ref:  NCRP Acknowledgement No. ______ / Helpline 1930 complaint dated ______

Sir/Madam,
   During investigation of the above case it is established from banking
   transaction records that funds defrauded from the complainant's account
   No. <account> (<IFSC>), amounting to ₹<amount>, were layered through
   intermediary accounts and credited to the following accounts maintained
   with your bank:

   | #  | Account Number | IFSC     | Amount (₹)  | Disputed transaction                                  |
   |----|----------------|----------|-------------|-------------------------------------------------------|
   | 1  | <account>      | <IFSC>   | <amount>    | ID <txn id>, dated <dd-mm-yy hh:mm:ss>, for ₹<amount>, from <acct> |

                                  TOTAL: ₹<total>

   You are hereby directed to:
   1. Place an immediate DEBIT FREEZE / LIEN to the extent of the above
      amounts and intimate the balance available at the time of action.
   2. Furnish KYC documents, account-opening forms and photographs.
   3. Furnish statements for <dataset start> to <dataset end>.
   4. Furnish registered mobile number, e-mail ID, and login/transaction
      IP logs with device details for the said period.
   5. Preserve CCTV/ATM footage of withdrawals, if any.

   Compliance is sought within 24 hours, the matter being time-sensitive
   and involving continuing dissipation of defrauded funds.

                                          Yours faithfully,
                                          ( _____ )
                                          Inspector of Police
                                          Cyber Crime Branch, Indore

Annexure-I : Chronology of layered fund movement (system-generated)
Annexure-II: Methodology & limitations note
Generated by ABHEDYA-CHAKRA · Dataset sha256:<fp> · n/n facts verified
Computer-generated draft — to be verified and signed by the I.O.
```

Disputed-transaction citation format is normative (D8): every disputed transaction is cited as `Transaction_ID` **plus** date-time **plus** amount **plus** counterparty. Where the ID occurs on multiple source rows, a footnote records: "Transaction ID appears on source records; the record cited here is identified by the date, amount and counterparty stated above."

21.4 Implementation notes [DEC]

Jinja2 templates with `autoescape`; all facts arrive pre-formatted from `format_evidence()` (Indian digit grouping, `DD-MM-YYYY HH:MM:SS`). PDF via WeasyPrint (A4, 20 mm margins, repeating table headers, page numbers, footer with fingerprint); DOCX via python-docx. Templates in `docs/templates/`. **Golden-file tests** assert byte-identical rendering from a fixed EvidenceObject. Both documents generate with `use_llm=false` by default.

22. Non-Functional Requirements

Targets are from the PS and are unchanged. v2.1 changes only the measurement basis: every benchmark is run on the actual supplied file, and every "Actual" cell stays blank until measured. Never fabricate a benchmark result.

ID	Requirement	Target	Actual	Measurement	Owner
NFR-01	Ingestion throughput **[PS]**	2,000,000 rows ≤ **60 s** on 16 GB RAM	_____	`make bench-ingest`, 3 runs, median	M1
NFR-02	Multi-hop query **[PS]**	4-hop trace ≤ **2 s** p95	_____	`make bench-trace`, 100 random + 20 chain victims	M1
NFR-03	Local execution **[PS]**	Ingestion, graph compute, dashboard render work **offline**	_____	Full demo with Wi-Fi disabled — rehearsed	M1
NFR-04	UI responsiveness **[PS]**	500+ nodes / 1,500+ edges interactive	_____	Chrome perf profile; pan/zoom ≥ 30 fps	M2
NFR-05	Memory ceiling	≤ **6 GB** ingest peak, ≤ **2.5 GB** steady	_____	`psutil` sampler in the bench harness	M1
NFR-06	Cold start	artifacts → serving ≤ **5 s**	_____	Startup log timestamps	M1
NFR-07	Document generation	≤ **2 s** without LLM, ≤ **10 s** with	_____	Endpoint timing	M4
NFR-08	Deployment simplicity	`make setup && make ingest && make run`	_____	Fresh-clone test on a second laptop	M4
NFR-09	Determinism	identical input → identical output	_____	Golden-file tests	M3
NFR-10	Crash-free demo	0 unhandled exceptions in a 10-min run	_____	Error boundary + structured logging	All
NFR-11	**Row conservation** (new)	`rows_out == 2,000,000`	_____	Ingest assertion (D1)	M1
NFR-12	**Profile agreement** (new)	`make profile-check` exits 0	_____	§7.2 expectation table	M3

22.1 Stage-level measurements required [DEC — new in v2.1]

`make bench` records each of these separately so a regression can be localised rather than guessed at:

Stage	Target	Actual
Profiling pass (FR-A05)	≤ 90 s	_____
CSV parse	—	_____
Normalize + encode	—	_____
Persist to DuckDB	—	_____
Feature generation (dual-sided)	—	_____
CSR graph build (E ≈ 2.0 M × 2)	—	_____
MRI computation (≈24.9k accounts)	—	_____
Layer computation (per trace)	—	_____
Total ingest	≤ 60 s	_____

Stage	Target	Actual
Peak RSS during ingest	≤ 6 GB	____
4-hop trace p50 / p95 / p99	p95 ≤ 2 s	____
Evidence retrieval (DuckDB)	—	____
Graph render (500n/1500e)	interactive	____
Case diary generation	≤ 2 s / ≤ 10 s	____

Rule: a blank cell is an honest answer; an estimate written as a measurement is a defect. Results are written to `docs/benchmarks.json` by the harness and copied into the README and the final slide from there, never typed by hand.

---

23. Security & Privacy Requirements

ID	Requirement	Implementation
SR-01	No unnecessary external transmission of raw transaction data	No outbound HTTP except optional localhost Ollama; a `STRICT_OFFLINE=1` flag hard-blocks all sockets except loopback
SR-02	Core processing remains local	Single-binary-ish local stack; no cloud SDKs in `requirements.txt`
SR-03	AI prompts contain only required evidence	Prompt builder accepts only the EvidenceObject summary; a unit test asserts no raw row ever reaches the prompt
SR-04	Generated documents validated against source data	§20.3 validator, mandatory, non-bypassable
SR-05	Account numbers and transaction IDs are never invented	V1/V2 + the LLM-no-digits rule (V8)
SR-06	Traceability between claims and source transactions	Every `TransactionEvidence` carries `source_row_id`; the annexure ledger maps every claim to a row
SR-07	Data at rest stays in the workspace	Artifacts under `data/`, gitignored; README warns against committing real data
SR-08	Masked display mode	Toggle renders accounts as `XXXX XXXX 5610` for screenshots/slides (shows privacy awareness to judges)
SR-09	No PII beyond the dataset	We never attempt to resolve account holders' identities
SR-10	Input hardening	Account IDs regex-validated; all SQL parameterised; file paths sandboxed to `data/`

24. Evaluation Strategy & Scoring Alignment

24.1 Official judging structure [PS] — unchanged

Criterion	Weight	What judges do
Blind victim query	**40%**	Provide 5 unannounced victim account IDs; expect trace, L1, L2, L3, real-time visualisation
Detection accuracy	**30%**	Evaluate against **1,500 injected mule accounts + 23,500 regular accounts**
Court-ready output	**20%**	Quality, factual accuracy and readiness of the S.94 BNSS notice and case diary
Architecture & engineering	**10%**	Code quality, memory management, deployment simplicity

24.2 Official evaluation population vs observed dataset profile [DEC — new in v2.1]

These are two different things and the PRD, the slides and the README must not conflate them.

Source	The problem statement	Preliminary profiling of `VoidHacks8_MuleAccount_2M_Transactions.csv`
Accounts	1,500 mule + 23,500 regular = 25,000	≈24,873 unique accounts
Labels	Described as existing for evaluation	**Not in our possession**
Status	Authoritative for how we will be judged	Authoritative for what we build against

The observed account count is within ≈0.5% of the PS-described population, which is **consistent with** the supplied file being the evaluation population. **It does not prove it, and we will not claim it does.**

Ground-truth label availability

```
GROUND-TRUTH LABEL AVAILABILITY : UNKNOWN — pending mentor confirmation (§29 Q1)
LABELS IN OUR POSSESSION        : NONE
CURRENT EVALUATION REGIME       : HEURISTIC SELF-VALIDATION
```

Every metric we publish carries its regime. We will not state or imply that we have the 1,500 labels unless they are actually provided.

Development fallback (all four run regardless): 1. **Synthetic mule injector** — inject known chains into a copy of the real file; tune on seed 1, report on seed 2. 2. **Manually validated pattern cases** — hand-built fixtures for each rule card and each layer case (§16.5). 3. **Unsupervised / heuristic validation** — score distribution shape, rank stability under threshold perturbation, manual review of the top-N. 4. **Blind victim trace tests** — the drill in §24.3, which is what the 40% criterion actually measures.

24.3 Blind-test readiness drill [DEC]

Run five times before the feature freeze, timed, with a teammate playing judge, on account IDs **not** previously tested:

```
Judge names an account ID  →  type it  →  Enter
   loading skeleton appears
   KPIs + graph rendered, layers colour-coded          ← record the time
   narrate the finding in one sentence
   click the top terminal node → inspector shows the combined evidence
   click FREEZE REQUISITION → PDF on screen containing that account
```

Any failure on any of 20 pre-tested random IDs is a **P0 bug**. Expected edge cases and their required behaviour are in §25 E.

24.4 Internal detection metrics

Tracked in `docs/EVALUATION.md`: precision, recall, F1, FP count, FN count, per-layer classification accuracy, PR curve, confusion matrix, top-10 false positives with reason lists, results split by injection difficulty, and the runtime of the full scoring pass. Two operating points are reported — **investigative (MRI ≥ 60)** and **actionable (MRI ≥ 80)** — because a false positive that reaches a freeze notice costs an innocent person access to their money.

24.5 Benchmark table for the README and final slide

Benchmark	Target	Actual	Status
Ingest 2,000,000 rows	≤ 60 s	__	__
Peak RSS during ingest	≤ 6 GB	__	__
4-hop trace (p95)	≤ 2 s	__	__
Account search (p95)	≤ 1 s	__	__
Graph render 500n/1500e	smooth	__	__
Case diary generation	≤ 10 s	__	__
Detection F1 (regime: ___)	–	__	__

Hardware: _____ · Dataset: VoidHacks8_MuleAccount_2M_Transactions.csv
sha256: _____ · Evaluation regime: _____

Fill from `docs/benchmarks.json`. **Leave blank until measured.**

---

25. Test Plan & Acceptance Matrix

25.1 Test layers

Unit — normalization edge cases, CSR correctness against a NetworkX reference on a small fixture, each rule card, matched pass-through arithmetic, MRI monotonicity and caps, validator V1–V12. **Golden file** — fixed trace input → expected JSON; fixed EvidenceObject → expected document text. **Integration** — ingest a fixture → investigate an injected victim → assert the L1/L2/L3 sets match the injection. **Performance smoke** — fails CI if ingest > 75 s or p95 trace > 2.5 s. **Adversarial** — see group E.

25.2 Dataset-specific tests [DEC — new in v2.1]

A · Ingestion

#	Test	Pass condition
A1	Row conservation	`rows_out == rows_in == 2,000,000`, asserted at every stage
A2	No silent row loss	Sum of per-stage deltas = 0; any drop is logged with a reason and fails the run
A3	Duplicate `Transaction_ID` survival	All rows sharing a duplicated ID are present and individually addressable
A4	ID-repeat histogram	`profile.json` reports `{repeat_count → n_ids}`; affected-row count is explicit, not inferred
A5	Exact-duplicate detection	Rows identical across all 11 fields are flagged, counted and retained
A6	Traceability	For 1,000 sampled rows, `source_row_id` round-trips: CSV line → DB row → API → document citation
A7	Aggregate vs raw consistency	Account aggregates exclude exact-duplicate secondaries (D6); the raw table shows all rows; the UI explains any difference
A8	Profile agreement	`make profile-check` passes against §7.2, or fails with a named mismatch

B · Account graph

#	Test	Pass condition
B1	Account count	≈24,873 unique accounts (±1%), matching `profile.json`
B2	Deterministic indexing	Same CSV → identical account→index mapping across runs
B3	Dual adjacency	Both `out_*` and `in_*` CSR arrays exist and are mutually consistent (every forward edge has its reverse)
B4	Edge-count sanity	`E` matches `profile.json → unique_pairs` (±1%); memory within the §13.1 budget
B5	Degree distribution	p50/p90/p99/max match the profiler; hub threshold derived from p99, not hard-coded
B6	Isolated/self-loop handling	Self-loops and zero-degree accounts do not crash traversal

C · Detection

#	Test	Pass condition
C1	High fan-in identified	Synthetic collector fires R2 with a reason naming senders and banks

#	Test	Pass condition
C2	High fan-out identified	Synthetic distributor fires R3 with a reason naming the split
C3	Matched pass-through	FIFO matching produces the expected ratio on hand-computed fixtures; `Σ matched ≤ Σ credits` and `≤ Σ debits`
C4	Pass-through edge cases	Each row of the §14.2 edge-case table has a test
C5	Terminal indicators	Signals detected from configurable lists; changing `config.py` changes the result
C6	**Combined-role account**	An account with high in-degree **and** high out-degree is scored on both; neither side is dropped
C7	**No single-feature verdict**	An account whose only positive evidence is a terminal signal cannot exceed `TERMINAL_SIGNAL_SOLO_CAP`, and cannot reach CRITICAL
C8	**Volume is not guilt**	A high-volume account with low pass-through receives the −15 suppression and stays below HIGH
C9	Reason-string hygiene	No generated reason matches the bare-signal pattern (e.g. a device name immediately followed by a mule conclusion)
C10	Co-occurrence discount	With `observed_cooccurrence_rate = 1.0`, C5 equals the single-signal score (no quadruple counting)

D · Layer classification (fixtures from §16.5)

#	Case	Expected
D1	Account only receives	Not auto-L3; absence of outgoing contributes at most `w12`
D2	Account receives **and** distributes	Classified on dominant role; both scores reported
D3	High fan-in **and** high fan-out	Role margin reported; no crash, no arbitrary tie-break
D4	Exactly 3 downstream	L2, inside the PS band
D5	Exactly 7 downstream	L2, inside the PS band
D6	More than 7 downstream	L2 with the wide-fan-out reason (R3b)
D7	Terminal with signals	L3, high confidence, reason combines flow + signal
D8	Terminal with **no** signal, strong sink	L3, medium confidence, reason names the absence of signals
D9	Same account in two victim traces	Different case-relative layers permitted; global modal role unchanged
D10	Confidence calibration	Mean confidence of correct assignments > that of incorrect ones on fixtures

E · Trace

#	Case	Expected
E1	Victim → L1 → L2 → L3	Full 4-hop chain, correct layers, p95 ≤ 2 s
E2	Terminal reached at hop 2	Short chain classified correctly; not forced to depth 3
E3	Victim → intermediary → terminal	Intermediary labelled L2/INTERMEDIARY, not silently skipped
E4	**Branch explosion**	With a 72 h window the result is capped and `truncated=true` with a reason; **the UI never attempts to render the whole graph** (§7.5)
E5	Hub account in the path	Entered, not expanded; labelled AGGREGATOR; excluded from freeze list

#	Case	Expected
E6	**Duplicate `Transaction_ID` on a traced edge**	Edge resolves to the correct `source_row_id`; document citation disambiguates (D8, V11)
E7	Account with no downstream flow	Upstream fallback offered; no empty canvas
E8	Non-existent account	Friendly error + fuzzy suggestion
E9	Determinism	Identical parameters → byte-identical result
E10	Window sensitivity	1 h / 6 h / 24 h / 72 h produce monotonically growing results; documented in the UI

F · Documents

#	Test	Pass condition
F1	All nine PS case-diary elements present	Checklist passes
F2	Hallucination test with a lying LLM stub	Validator rejects, retries, falls back to template; **0 fabricated facts**
F3	Duplicate-ID citation	Any cited duplicated ID carries date-time + amount + counterparty (V11)
F4	Row-level re-verification	V12 re-reads every cited `source_row_id` from DuckDB and matches field-by-field
F5	Methodology annexure	States attribution convention, window, hop limit, case-relative layers, signals-as-evidence

25.3 Acceptance matrix (requirement → test)

Req	Acceptance test	Pass condition
FR-A01	`make bench-ingest`	≤ 60 s, no OOM, A1–A2 pass
FR-A02	Normalization report	0 rows dropped; enum domains match the profiler
FR-A03	`/search/account` ×100	p95 < 1 s
FR-A04	Manifest inspection	Fingerprint + duplicate stats + config snapshot present
FR-A05	`make profile && make profile-check`	profile.json complete; §7.2 agrees or fails loudly
FR-B02	Feature table inspection	Both sides populated for 100% of accounts
FR-B03	`/risk` on 20 accounts	0–100, components sum correctly, ≥1 reason, caps respected
FR-B04	Pass-through fixtures	C3, C4 pass
FR-B05/06/07	Injected 3-layer chain	L1/L2/L3 sets match the injection
FR-B08	`make bench-trace`	p95 ≤ 2 s at depth 4
FR-C02	600-node/1,800-edge render	Interactive; no main-thread block > 200 ms
FR-C03	Timeline drag across the window	Re-filter ≤ 150 ms; counts match SQL
FR-C04	Subgraph export	CSV row count == edge source-row count; `source_row_id` present
FR-D01/02	Document generation	F1, F3, F5 pass

Req	Acceptance test	Pass condition
FR-D03	F2 hallucination test	0 fabricated facts
NFR-03	Full demo with networking disabled	Everything works
NFR-11	Row conservation	Exactly 2,000,000 rows

25.4 Definition of Done (per module)

Code merged · unit tests pass · endpoint in OpenAPI · thresholds in `config.py` · appears in the demo script or is explicitly marked non-demo · README updated · no `TODO` on the demo path · **no estimated number presented as measured**.

---

26. Team Responsibilities & RACI

M1 — Technical Lead (you)

Owns: architecture, ingestion engine, CSR graph, traversal, performance, FastAPI integration, hard debugging, final merge authority. **Does not own:** UI styling, document wording, threshold tuning. Delegate these or you become the bottleneck.

M2 — Frontend

Owns: dashboard, search UX, Cytoscape canvas, timeline, node inspector, evidence basket, API client, empty/error states, demo polish.

M3 — Detection / Data

Owns: the dataset profiler and `make profile-check` (FR-A05) — the first deliverable of Phase 1, dual-sided feature engineering, rule cards, thresholds, MRI calibration, layer classifier tuning, ring/cycle detection, precision-recall evaluation, synthetic mule injector, `docs/EVALUATION.md`.

M4 — Forensics / Documentation / QA

Owns: Evidence Object, validator, Jinja templates, PDF/DOCX pipeline, document UI, QA test scenarios, README, screenshots, slide deck, demo rehearsal, submission package.

RACI (R=responsible, A=accountable, C=consulted, I=informed)

Deliverable	M1	M2	M3	M4
Dataset profiling & verification (FR-A05)	C	I	**A/R**	I
Ingestion pipeline	**A/R**	I	C	I
Graph + traversal	**A/R**	C	C	I
Feature extraction	C	I	**A/R**	I
MRI + thresholds	C	I	**A/R**	I
Layer classification	**A**	I	**R**	I
API contracts	**A/R**	C	C	C
Dashboard + graph UI	C	**A/R**	I	C
Timeline + subgraph	C	**A/R**	I	I
Evidence Object + validator	C	I	C	**A/R**
Case diary / requisition	I	C	I	**A/R**
Benchmarks	**A/R**	C	C	I
Evaluation metrics	C	I	**A/R**	C
README / slides / demo	C	C	C	**A/R**

Working agreements [DEC]

- **Contracts first:** M1 publishes stub API responses in the first 90 minutes so M2 and M4 are never blocked. Stubs return realistic fake data behind `MOCK=1`.
- **Branch per module**, PR to `main`, M1 merges. No direct pushes to `main` after Phase 3.
- **Thresholds live only in `config.py`.** M3 owns that file.

- **Standups at phase boundaries** (6 total), max 5 minutes, three questions: done / blocked / next.
- **Blocked > 20 minutes = escalate to M1 immediately.** No silent struggling at 3 a.m.
- **Sleep rotation:** two people sleep 2:00–5:00, two sleep 5:00–8:00. A team that is awake for 36 hours demos badly. This is a performance decision, not a kindness.

---

27. 36-Hour Execution Plan

v2.1 inserts the dataset profiler at the very front. Nothing downstream is tuned until the profiler has told us what is actually in the file (P8).

> Each phase has a **hard exit criterion**. If it isn't met, cut scope — do not extend the phase.

Phase 0 · 7:30–8:00 PM — Setup (30 min)

Repo created, everyone cloned, Python/Node installed, **CSV downloaded and checksummed**, `config.py` skeleton committed, API stub contracts agreed, roles confirmed. **Exit:** `make run` serves `/health` on every laptop; the CSV's SHA-256 is recorded.

Phase 1 · 8:00–11:00 PM — Foundation + Profiling first

Who	Task
M1	**Run the profiler before anything else (FR-A05).** Then: ingest pipeline, normalization, `source_row_id`, duplicate handling (D1–D9), account encoding, persist artifacts, **benchmark the real 2M file**
M3	**Own `make profile-check` against §7.2.** Reconcile every preliminary figure; resolve the duplicate-ID histogram; produce the `signal_cooccurrence` cross-tab (§7.4); report degree distribution for the hub threshold (§13.3)
M2	Vite + React + Tailwind scaffold, layout shell, API client, search bar on the `MOCK=1` stubs
M4	Repo hygiene, README skeleton, document-template scaffold, Jinja setup

Exit criteria (all three): 1. ✔ 2,000,000 rows ingested, **row conservation asserted**, searchable 2. ✔ `profile.json` written and `make profile-check` run — §7.0 status block replaced with a verified stamp, or discrepancies logged as P0 3. ✔ Ingest time recorded (not estimated)

> **If the profiler contradicts §7.1, stop and update the PRD §7 and §33 before tuning anything.** A wrong assumption propagated into detection costs more hours than the correction.

Phase 2 · 11:00 PM–5:00 AM — Core Detection

Who	Task
M1	CSR build sized for E ≈ 2.0 M, `trace()` with the **recalibrated window/min_value/topk** (§17.2), hub guard from the measured p99, `/investigate`, trace benchmark
M2	Cytoscape canvas, dagre layered layout, node/edge styling, KPI strip, wired to real `/investigate`
M3	**Dual-sided feature computation (both directions, every account)**, matched pass-through (§14.2), rule cards R1–R10, MRI v1 with the dominance cap, first threshold pass
M4	EvidenceObject with `source_row_id` provenance, builder from a trace result, validator V1–V6 + V11/V12, first case-diary render

Checkpoint 2:00 AM: end-to-end "account ID in → JSON trace out" works, however ugly. **Exit:** ✔ Victim ID → 4-hop trail → L1/L2/L3 with confidence → MRI, through the API, under 2 s.

Phase 3 · 5:00–10:00 AM — Investigation UI

Who	Task
M1	Subgraph, timeline, search endpoints; trace optimisation; whatever Phase 2 exposed
M2	**Full dashboard**: node inspector with **both-sided flow and the case-relative role note**, layer legend, timeline slider + histogram, loading/empty/error states
M3	Layer classifier (§16) with the ten inputs and confidence; §16.5 fixtures; threshold tuning
M4	Case diary → HTML → PDF; document preview modal; methodology annexure

Exit: ✔ A judge could sit down and use it unaided.

Phase 4 · 10:00 AM–7:00 PM — Intelligence + Evidence

Who	Task
M1	Ring detection, cycles, upstream trace, exports with `source_row_id`, `/eval/metrics` with the `regime` field, perf pass
M2	Subgraph isolation, evidence basket, timeline playback, export buttons, Ring Board, **window-preset control (1 h/ 6 h/24 h/72 h)**
M3	MRI calibration (**labelled heuristic self-validation**), suppression rules, FP analysis, `docs/EVALUATION.md`
M4	Freeze requisition per bank with duplicate-ID disambiguation, DOCX, LLM prose + V7–V10, QA sweep

Checkpoint 2:00 PM: MVP complete. **Checkpoint 5:00 PM: first full timed demo rehearsal. Exit:** ✔ Case diary + freeze requisition generated from a blind account, validated, exported as PDF.

Phase 5 · 7:00 PM–3:00 AM — Optimization & Hardening

Query speed · memory · graph rendering · detection precision · false-positive reduction · **branch-explosion testing at all four window presets** · the blind drill ×5 · adversarial inputs · fresh-clone deployment test · README with **measured** numbers · screenshots. **Exit:** ✔ 20 random account IDs tested; all correct, fast and legible.

Phase 6 · 3:00–5:00 AM — 🚨 FEATURE FREEZE (final mentorship window)

No new features. No refactors. No dependency changes. Bug fixes on the demo path, documentation, slides, rehearsal. **Use the mentors for §29 — especially Q1 (labels), Q3 (thresholds) and Q10 (layer evaluation).** Tag and back up: `git tag v1.0-freeze && git bundle create backup.bundle --all`, plus USB and cloud copies.

Phase 7 · 5:00–8:00 AM — Finalisation

Final test pass · clean-clone packaging check · screenshots · README final · PPT final · demo rehearsed 3 more times including the failure path · backups · submission package. **08:00 completion · 08:00–08:30 submission.**

27.1 Priority shifts introduced by the real dataset [DEC — new in v2.1]

Priority	Why	Whose time
⬆ Profiling and verification, before tuning	Preliminary figures are unverified; a wrong assumption compounds	M3, Phase 1
⬆ Duplicate-ID / provenance plumbing	Affects data model, evidence, documents and exports — expensive to retrofit	M1 + M4, Phase 1–2
⬆ Branch-explosion control	At a 72 h window the trace returns the whole graph (§7.5) — this is a correctness issue, not a performance nicety	M1, Phase 2
⬆ Behavioural detection of L1 and L2	Terminal signals touch 0.13% of rows and cannot find L1/L2 (§7.4)	M3, Phase 2–4
⬇ Terminal keyword enrichment	The list is already complete; adding synonyms adds nothing	—
⬇ Edge-aggregation optimisation	≈99.8% of rows are unique pairs, so aggregation saves almost nothing (§7.6)	—
➡ Everything else	Unchanged from v2.0	—

28. MVP Ladder, Risks & Contingencies

28.1 MVP ladder

🔴 MUST HAVE — without these we do not place

```
2M ingestion → account search → victim investigation → 4-hop trace
→ L1/L2/L3 detection → MRI → interactive graph
```

🟠 SHOULD HAVE — these win the remaining 20–30%

```
Timeline filter · subgraph isolation · export · case diary · freeze requisition
```

🟢 NICE TO HAVE — only if everything above is green

```
LLM prose · ring board · cycle detection · playback animation · eval dashboard
```

This is our execution strategy, not an official judging hierarchy beyond the published criteria.

28.2 Risk register

#	Risk	Prob	Impact	Mitigation	Trigger → Contingency
R1	2M ingest too slow	Med	🔴	DuckDB native CSV reader, int encoding, no Python loops	>60 s at Phase 1 exit → switch to Parquet pre-conversion + streaming; worst case demo on 500k subset and say so
R2	RAM exhaustion	Low	🔴	int32/uint8 encoding, streaming mode, aggregated edges	RSS > 10 GB → enable `STREAMING=1` chunked path
R3	4-hop query > 2 s	Med	🔴	CSR + pruning + precomputed features	>2 s → reduce `topk`, cache last 50 traces in an LRU, pre-warm the 5 demo IDs
R4	Graph blow-up via hub	**High**	🔴	Hub guard §13.3, value pruning, node cap	>800 nodes → auto-collapse leaves, show "expand" banner
R5	False positives tank precision	Med	🔴	Suppression rules, threshold sweep, FP review	Precision < 0.7 → raise MRI threshold, add volume floor
R6	AI hallucination	Low (by design)	🔴	Template-first, validator, no-digits rule	Any validator failure in testing → ship `use_llm=false` and present that as the correct engineering choice
R7	Browser freeze during demo	Med	🟠	Element cap, batched updates, texture-on-viewport	Freeze → `[Simplify view]` button reduces to top-100 nodes
R8	LLM/API unavailable	Med	🟠	Fully functional offline template path	Ollama slow → toggle off; demo unaffected

#	Risk	Prob	Impact	Mitigation	Trigger → Contingency
R9	Team bottleneck on M1	**High**	🟠	Mock API in first 90 min, clear ownership, 20-min escalation rule	M1 blocked → M3 takes detection autonomously, M2 continues on mocks
R10	Last-minute breakage	Med	🔴	Feature freeze at 3 AM, tagged backup, rehearsals	Broken → `git checkout v1.0-freeze` and demo that
R11	Blind ID isn't a victim / has no trail	Med	🔴	Upstream-trace fallback + informative empty state (§18.5, §17.5)	Empty result → one click to "view upstream sources"
R12	Dataset schema differs from spec	Low	🔴	Column-name normalizer with alias map + tolerant parser	Mismatch → alias map edit, 5-minute fix
R13	Laptop dies / projector fails	Low	🔴	Backup laptop with full artifacts, recorded 3-min demo video	Hardware fail → play the video, keep talking
R14	Everyone is exhausted by judging	**High**	🟠	Enforced sleep rotation, rehearsed script	—
R15	**Preliminary dataset figures turn out wrong** (new)	Med	🔴	`make profile-check` in Phase 1 before any tuning (P8)	Mismatch → correct §7 and §33, re-derive thresholds, before writing detection code
R16	**Branch explosion returns the whole graph** (new)	**High**	🔴	6 h default window, value floor, top-K, hub guard (§7.5, §17.2)	Node cap hit → `truncated=true` + collapse banner; never render uncapped
R17	**Duplicate `Transaction_ID` corrupts a citation** (new)	Med	🔴	`source_row_id` provenance, V11/V12 validators, mandatory disambiguation (D8)	Ambiguous citation → document generation fails loudly rather than emitting it
R18	**A single injected signal dominates the risk score** (new)	Med	🟠	Co-occurrence discount, `SINGLE_COMPONENT_CAP`, `TERMINAL_SIGNAL_SOLO_CAP` (§15)	Score distribution spikes at one value → inspect C5, re-run the cap tests

28.3 The "everything is on fire" plan

If at 2:00 PM on day 2 the MVP is not complete: freeze detection at whatever quality it has, drop the LLM, drop rings/cycles/playback, and spend all remaining time on **(a) the blind-query path and (b) the two documents**. Those two things are 60% of the score.

---

29. Open Questions & Standing Assumptions

Ask these in the first mentor window. Until answered, build against the assumption column — no question blocks the build. We do not invent answers, and we do not record a mentor's answer here until it has actually been given.

#	Question to mentor	★	Standing assumption	Status	Cost if wrong
1	**Are the 1,500 ground-truth mule labels available during development?**	★	**No.** We hold no labels. All tuning is heuristic self-validation (§15.4, §24.2).	**OPEN**	Low — the injector is useful either way
2	How exactly are precision and recall calculated — account-level, layer-level, or per victim?	★	Account-level binary over the ≈24.9k accounts.	**OPEN**	Medium — retune the operating point
3	Are 90% / 3–15 min / 3–7 accounts **fixed** requirements or tunable heuristics?	★	Tunable; defaults set exactly to the PS values, all in `config.py`.	**OPEN**	Low — one-line change
4	What operations count inside the 60-second ingestion benchmark — does the FR-A05 profiling pass count?	★	Parse + normalize + persist + index + features, i.e. everything up to "searchable". **Profiling is reported separately** and excluded.	**OPEN**	Low — we report both numbers
5	Does the 2-second 4-hop target mean API time or end-to-end including render?	★	End-to-end perceived. We measure and publish both.	**OPEN**	Low
6	Will blind-test victim accounts always have a known mule trail?	★	Not guaranteed → upstream fallback + graceful empty state (§17.5).	**OPEN**	**High if wrong and unhandled**
7	Is there an official template for the case diary and freeze requisition?	★	No → the standard Indian police format of §21, template-driven and swappable in ~10 minutes.	**OPEN**	Medium
8	May an external LLM API be used, given the local-processing requirement?	★	No → local Ollama, with a complete non-LLM path.	**OPEN**	Low — we are strictly safer
9	Is the legal formatting our responsibility or supplied?		Ours; cite S.94 BNSS (≡ S.91 CrPC) with an IO-verification disclaimer.	**OPEN**	Low
10	How are L1/L2/L3 labels evaluated — per victim, or globally per account?	★	Per victim (layers are case-relative, §2.5). We **also** store a global modal role to cover the other interpretation.	**OPEN**	Medium — covered both ways
11	Is the dataset timezone IST, and are timestamps naive?		Naive local time == IST; displayed as supplied, **never converted**.	**OPEN**	Low — no conversion exists to be wrong
12	May we show a recorded fallback video if hardware fails during judging?		Yes, but we prepare one regardless.	**OPEN**	Low
13	**Are the duplicate `Transaction_ID` values intentional, or an artefact of dataset generation?** (new in v2.1)	★	Treated as **real data to be preserved, not cleaned** (§11.3). We report them; we never delete them.	**OPEN**	Low — preservation is safe under either answer

#	Question to mentor	★	Standing assumption	Status	Cost if wrong
14	**Should exact-duplicate rows be counted once or twice in account aggregates?** (new in v2.1)	★	Counted **once** for scoring (D6); **both retained** in evidence; the difference is disclosed in the UI.	**OPEN**	Low — one config flag
15	**Do the terminal indicators (foreign IP, automation device, crypto/wallet narration) co-occur on the same injected transactions?** (new in v2.1)	★	**Yes, provisionally** (§7.4) — scored as one composite with a co-occurrence discount. The profiler's cross-tab settles it empirically in Phase 1.	**OPEN — resolvable by us**	Medium — would otherwise quadruple-count one fact
16	**Is the supplied CSV the same population the 1,500/23,500 evaluation describes?** (new in v2.1)		Assumed consistent (≈24,873 observed vs 25,000 described) but **not claimed as fact** (§24.2).	**OPEN**	Low
17	**Will judges supply victim IDs from this file, or from a held-out file?** (new in v2.1)	★	From this file. If a held-out file arrives, ingestion is file-agnostic and the profiler runs first.	**OPEN**	Medium — mitigated by design

Q15 is the one question we can answer ourselves. It is a Phase-1 profiler task, not a mentor dependency.

---

30. Demo Script & Submission Package

30.1 Five-minute demo (rehearse until it's muscle memory)

Time	Beat	Say / Do
0:00–0:30	**The problem**	"A victim loses ₹2.5 lakh to a digital-arrest scam. The bank sends the IO a CSV with two million rows. Today, finding where that money went takes days. The money is gone in hours."
0:30–1:00	**The system**	Show the status screen: "2,000,000 transactions ingested in <measured> seconds, fully indexed, entirely on this laptop — Wi-Fi is off." (Turn Wi-Fi off on stage. It lands.) **Read the real number off the screen; never rehearse a figure you have not measured.**
1:00–2:00	**Blind query**	"Give me any victim account." Type it, Enter. **The live `query_ms` readout** appears on screen. "₹2,50,000 siphoned. One Layer-1 collector. Five Layer-2 distributors. Eleven Layer-3 cash-out nodes."
2:00–2:45	**The evidence**	Click the collector: "98.8% of the money left within 4 minutes 10 seconds, split five ways — that's the pass-through signature." Click a terminal node: "IP 185.22.14.90, device Linux_Script, narration 'USDT P2P'. This is the exit."
2:45–3:15	**Timeline**	Scrub: "The entire laundering operation completed in 45 minutes. Here it is, minute by minute." Hit Play.
3:15–4:15	**Court-ready output**	Click Freeze Requisition. PDF appears. "Section 94 BNSS notice, per bank, with exact account numbers, IFSC codes and disputed transaction IDs — ready for the IO's signature." Then the case diary.
4:15–4:45	**Anti-hallucination**	Point at the badge: "47 of 47 facts verified against the source dataset. The model never writes a number — the database does. That's enforced by a validator, not by a prompt."
4:45–5:00	**Close**	Benchmark slide, populated from `docs/benchmarks.json`: measured ingest time, measured p95 trace, detection F1 **with its evaluation regime stated**, fully offline. "Victim ID to signed freeze notice in under two minutes."

Demo rules: one person drives, one person narrates, two stay silent. Browser at 1366×768, zoom 100%, one tab, notifications off, dark mode, laptop plugged in. Have the dataset pre-ingested — **never ingest live** (show the recorded timing instead; run a live 200k ingest only if asked).

30.2 Submission package checklist

- [ ] GitHub repo, public, clean history, no large data files committed
- [ ] `README.md`: one-paragraph pitch, screenshots/GIF, 3-command setup, architecture diagram, **benchmark table with measured numbers only (blank if unmeasured)**, dataset profile summary, team, licence
- [ ] `docs/PRD.md`, `docs/ARCHITECTURE.md`, `docs/EVALUATION.md`, `docs/DEMO.md`
- [ ] Sample outputs: `samples/case_diary.pdf`, `samples/freeze_requisition.pdf`, `samples/subgraph_export.csv`, `samples/profile.json`
- [ ] Slide deck (8–10 slides: problem, approach, architecture, detection, anti-hallucination, results/benchmarks, demo screenshots, roadmap, team)
- [ ] 3-minute demo video (fallback + submission asset)
- [ ] `make setup && make ingest && make run` verified on a **fresh clone, second machine**
- [ ] Tagged release `v1.0`, plus offline backup (USB + cloud)

31. Appendices

A. Glossary

Mule account — an account used to receive and forward criminal proceeds. **Layering** — moving funds through intermediaries to obscure origin. **Smurfing/structuring** — splitting amounts to avoid detection. **Collector (L1)** — role descriptor for an account whose dominant behaviour in a victim's trail is aggregation from many senders; it normally also sends onward. **Distributor (L2)** — role descriptor for an account that receives from upstream and splits onward to ~3–7 accounts. **Terminal (L3)** — apparent cash-out endpoint, established by evidence rather than by absence of outgoing rows. **Case-relative role** — a role valid for one victim's trail; the same account may hold a different role in another case. **Pass-through ratio** — share of credits temporally matched to onward debits (§14.2). **Hold time** — interval between a credit and its matched debit. **MRI** — Mule Risk Index, 0–100. **CSR** — Compressed Sparse Row adjacency. `source_row_id` — immutable 0-based CSV row ordinal; the provenance anchor. `evidence_row_id` — `fingerprint:source_row_id`, globally unique. **NCRP** — National Cyber Crime Reporting Portal. **1930** — national cyber-fraud helpline. **BNSS** — Bharatiya Nagarik Suraksha Sanhita, 2023. **S.94 BNSS** — summons/ written order to produce documents (≡ S.91 CrPC).

B. Configuration constants — `backend/abhedya/config.py`

Single source of truth. Values changed in v2.1 are marked. Nothing here may be inlined elsewhere.

```
# ── Dataset expectations (FR-A05; asserted by make profile-check) ──────
EXPECTED_ROWS            = 2_000_000          # new in v2.1
EXPECTED_COLUMNS         = 11
EXPECTED_ACCOUNTS        = 24_873             # ±1%, [DATA-PRELIM]
EXPECTED_DATE_RANGE      = ("2026-09-15", "2026-09-29")   # [DATA-PRELIM]
PROFILE_TOLERANCE        = 0.01

# ── Duplicate / provenance policy (§11.3) ──────────────────────────────
PRESERVE_ALL_ROWS        = True               # D1 — never flip this to False
TXN_ID_IS_UNIQUE         = False              # changed in v2.1
DEDUPE_EXACT_IN_AGGREGATES = True             # D6; raw views still show all rows

# ── Velocity / pass-through (FR-B04) ───────────────────────────────────
PASS_THROUGH_RATIO       = 0.90               # [PS]
VELOCITY_WINDOW_SEC      = (180, 900)         # 3–15 minutes [PS]
MIN_HOLD_SEC             = 180                # below this = instant_forward flag
PASSTHROUGH_MATCH_SEC    = 86_400             # FIFO matching horizon (new in v2.1)
NEAR_PASSTHROUGH_RATIO   = 0.80
NEAR_WINDOW_SEC          = 3_600
ATTRIBUTION_METHOD       = "FIFO"             # declared convention (new in v2.1)

# ── Role shapes (FR-B05/B06) ───────────────────────────────────────────
COLLECTOR_MIN_IN_DEGREE  = 5
DISTRIBUTOR_FANOUT       = (3, 7)             # [PS]
SPLIT_CONSISTENCY_MAX    = 0.35
DEGREE_BALANCE_MIN       = 0.20               # new in v2.1

# ── Terminal signals (FR-B07) — evidence, never verdicts ───────────────
FOREIGN_IP_PREFIXES      = {185, 194}         # [PS], configurable
AUTOMATION_DEVICES       = {"Web_Emulator", "Linux_Script"}   # [PS], configurable
CRYPTO_KEYWORDS          = {"p2p", "crypto", "usdt", "btc", "binance", "tether"}
WALLET_KEYWORDS          = {"wallet", "topup", "paytm", "phonepe", "mobikwik"}
TERMINAL_SIG_RATIO_MIN   = 0.30
SINK_RATIO_MIN           = 0.85
SINK_MIN_AMOUNT          = 10_000             # was 50_000 — median txn is ~₹899
TERMINAL_INDEP_BONUS     = 0.25               # new in v2.1, co-occurrence discount
TERMINAL_SIGNAL_SOLO_CAP = 45                 # new in v2.1, P10

# ── Risk scoring (§15) ─────────────────────────────────────────────────
W_PASSTHROUGH, W_VELOCITY, W_FANIN, W_FANOUT, W_TERMINAL, W_BEHAV = 25,20,15,15,15,10
SINGLE_COMPONENT_CAP     = 40                 # new in v2.1, dominance cap
BAND_EDGES               = (30, 60, 80)
FREEZE_MRI_MIN           = 80
FREEZE_HOLDING_MIN       = 2_000              # was 10_000 — recalibrated to amounts
SUPPRESS_MIN_AMOUNT      = 5_000
HIGH_VOLUME_SUPPRESSION  = -15                # new in v2.1, "busy is not guilty"

# ── Traversal (§17) ────────────────────────────────────────────────────
TRACE_DEPTH              = 4                  # [PS]
TRACE_WINDOW_SEC         = 21_600             # 6 h — was 72 h  (§7.5)
TRACE_WINDOW_PRESETS     = (3_600, 21_600, 86_400, 259_200)
TRACE_MIN_VALUE_ABS      = 100                # was 1_000 flat  (§7.1)
TRACE_MIN_VALUE_REL      = 0.002              # 0.2% of the initial amount
TRACE_TOPK_PER_NODE      = 15                 # was 25
RESPONSE_WINDOW_SEC      = 21_600             # "responds after funding" (§16.2)
HUB_DEGREE_FLOOR         = 500                # was a flat 2_000 cap
HUB_P99_MULTIPLIER       = 3.0                # cap = max(floor, p99 × multiplier)
HUB_PASSTHROUGH_MIN      = 25_000

# ── Layer classifier weights (§16.2) ───────────────────────────────────
COLLECTOR_MIN, DISTRIBUTOR_MIN, TERMINAL_MIN = 0.45, 0.45, 0.50

# ── UI / performance ───────────────────────────────────────────────────
GRAPH_NODE_CAP           = 800
GRAPH_EDGE_CAP           = 2_500
TRACE_P95_TARGET_MS      = 2_000              # [PS]
INGEST_TARGET_SEC        = 60                 # [PS]
```

C. PS-requirement → implementation traceability

PS requirement	Section	Artifact
Ingest 2M in ≤60 s @16 GB	§12, NFR-01	`ingest/`, `bench_ingest.py`
Pass-through ≥90% in 3–15 min	§14.2, R1	`detect/passthrough.py`, `detect/rules.py`
Collector = high fan-in, many senders	§14.3 R2, §16	`detect/rules.py`, `detect/layers.py`

PS requirement	Section	Artifact
Distributor = fan-out 3–7	§14.3 R3, §16	same
Terminal = wallet/crypto/185.x/194.x/Web_Emulator/ Linux_Script	§14.4, §11.2	`ingest/normalize.py`, `detect/rules.py`
Mule Risk Index 0–100	§15	`detect/scoring.py`
4-hop trace ≤2 s	§17, NFR-02	`graph/traverse.py`, `bench_trace.py`
Interactive graph 500+/1500+	§18.4, NFR-04	`GraphCanvas.tsx`
15-day timeline filtering	FR-C03	`TimelineSlider.tsx`, `/timeline`
Subgraph isolation + export	FR-C04	`/export/subgraph`
Case diary (9 elements)	§21.2	`docs/diary.py`
Freeze requisition, S.91/BNSS	§21.3	`docs/requisition.py`
No hallucinated accounts/amounts	§20	`evidence/validator.py`
Local/offline core	§9.2, NFR-03, SR-02	whole stack

D. Pre-demo checklist

```
[ ] Dataset pre-ingested; profile.json present and profile-check green
[ ] 20 test account IDs verified working (list taped next to the laptop)
[ ] Wi-Fi off — full flow still works
[ ] Browser 1366×768, zoom 100%, single tab, notifications off
[ ] Sample PDFs open in a second tab as backup
[ ] Demo video on the desktop as backup
[ ] Laptop plugged in; battery saver off; screen sleep disabled
[ ] Backup laptop booted with the same state
[ ] git tag v1.0-freeze pushed + USB copy + cloud copy
[ ] README benchmark table contains MEASURED numbers only
[ ] One driver, one narrator, agreed and rehearsed
```

32. Changelog v2.0 → v2.1

Driver: receipt of the actual hackathon dataset, `VoidHacks8_MuleAccount_2M_Transactions.csv`. **Principle:** refinement based on evidence, not rewriting for its own sake. The architecture, stack, terminology, requirement IDs, evaluation alignment and 36-hour structure of v2.0 are retained.

32.1 Verification status of this revision

Dataset figures in v2.1	Supplied by the team as **preliminary profiling**
Independently recomputed from the CSV by the document author?	**No — the file was not accessible during this revision**
What was done	Arithmetic consistency checks and derivation of the quantities that affect architecture (§7.0)
How figures become verified	`make profile` + `make profile-check` (FR-A05, §7.2) in Phase 1
Marker used	**[DATA-PRELIM]** on every unverified dataset figure

32.2 Section-by-section changes

Section	Change	Type
Header	Version 2.1; dataset named; **[DATA]** marker introduced	Modified
§2	**§2.4 added** — layers are role descriptors, not exclusive capabilities; **§2.5** keeps layers victim-relative	**Added**
§3.1	Principles **P7–P10** added (roles are descriptors · profile before you assume · never destroy a source row · a signal is evidence not a verdict)	**Added**
§7	**Rewritten** as Dataset Reality Check & Sizing: verification status, observed profile, profiler expectation table, duplicate-ID finding, terminal-signal co-occurrence finding, branch-explosion finding, edge-count correction	**Rewritten**
§8 Module A	**FR-A05 Dataset Profiling & Schema Discovery added**; FR-A01/A02/A04 updated for row conservation and discovered enums	**Added / modified**
§8 Module B	**FR-B02 Dual-Sided Account Features added**; FR-B05/B06/B07 reworded away from "only collects / only distributes"; requirement IDs shifted by one from FR-B02 onward	**Added / modified**
§8 Modules C, D	Node inspector shows both-sided flow and case-relative role; exports and citations carry `source_row_id`	Modified
§11	**Rewritten** — `source_row_id`, `evidence_row_id`, `row_hash`, duplicate flags, `terminal_signal_mask`; **§11.3 duplicate-ID policy D1–D9**	**Rewritten**
§12	Profiling stage [0] and duplicate-report stage [5b]; row-conservation assertions; dual-sided aggregation SQL; stage timings explicitly marked as estimates	Modified
§13	CSR resized for E ≈ 2.0 M (~128 MB both directions); `out_signal` array; **distribution-relative hub guard**	Modified
§14	**Rewritten** — dual-sided catalogue (~45 features); **matched FIFO pass-through** replacing ratio-of-totals, with a specified edge-case table; rule cards restated as role shapes; **§14.4 signals-as-evidence**; high-volume suppressor	**Rewritten**

Section	Change	Type
§15	Formula structure preserved; **C5 co-occurrence discount**; **dominance cap** and `TERMINAL_SIGNAL_SOLO_CAP`; `FREEZE_HOLDING_MIN` ₹10,000 → ₹2,000; calibration explicitly labelled heuristic self-validation	Modified
§16	**Rewritten** — ten classification inputs; forbidden single-signal rules; confidence decomposition; worked case table	**Rewritten**
§17	Algorithm unchanged; **window 72 h → 6 h**, **min_value flat ₹1,000 → max(₹100, 0.2%)**, **topk 25 → 15**, hub cap now p99-relative; `source_row_id` on every edge; budgets separated from measurements	Modified
§19	`/profile`, `/profile/duplicates`, `/row/{source_row_id}` added; `/transaction/{id}` returns a **list**; new error codes; `regime` field on `/eval/metrics`	Modified
§20	Evidence Object carries `source_row_id`, duplicate flags, both-sided flow, attribution method, trace params; **validators V11–V12**	Modified
§21	Specimens explicitly marked **illustrative**; dates moved inside the observed window; duplicate-ID citation format made normative; **Methodology & limitations annexure** added	Modified
§22	All "Actual" cells blank until measured; **NFR-11 row conservation**, **NFR-12 profile agreement**; stage-level measurement table	Modified
§24	**Official PS population separated from observed profile**; ground-truth availability stated as UNKNOWN; evaluation regime attached to every metric	Modified
§25	**Rewritten** — dataset-specific groups A–F including duplicate handling, combined-role accounts, single-feature prohibition, branch explosion, window sensitivity	**Rewritten**
§27	Profiler runs **first**; Phase-1 exit now requires `profile-check`; §27.1 priority shifts	Modified
§29	**Q13–Q17 added** (duplicate intent, exact-duplicate counting, signal co-occurrence, population identity, held-out file); every question marked **OPEN**	**Added**
§31 B	Config constants updated and annotated with what changed and why	Modified
§32, §33	This changelog and the consequences summary	**New**

32.3 What was deliberately not changed

Local-first architecture · Polars · DuckDB · NumPy CSR (no NetworkX in the hot path) · FastAPI · React + Vite · Cytoscape.js · timeline playback · evidence basket · EvidenceObject · anti-hallucination validation · deterministic template-first document generation · optional local Ollama for prose only · 4-hop value-weighted tracing · hub guard · ring detection · cycle detection · benchmark tooling · 36-hour phase structure · team RACI · mentor-question discipline · MRI weights (25/20/15/15/15/10) · band edges (30/60/80) · all PS targets and all judging weights.

32.4 Instructions withdrawn from v2.0

Withdrawn	Replaced by
"`Transaction_ID` ... assert unique; duplicates quarantined" (§11.1)	§11.3 D1–D9 — preserve every row; `source_row_id` is the key
"Unique pairs 300k–900k ... ~26 MB" (§7)	§7.6 — E ≈ 2.0 M, ~128 MB both directions
`TRACE_WINDOW_SEC = 72 h`	6 h default with UI presets (§7.5)
`TRACE_MIN_VALUE = 1000` flat	`max(₹100, 0.2% × initial_amount)`
`HUB_DEGREE_CAP = 2000` absolute	`max(500, p99_out_degree × 3)`

Withdrawn	Replaced by
`pass_through_ratio = out_amount/in_amount`	FIFO temporal matching (§14.2)
C5 as a blend of four independent signals	Composite with co-occurrence discount (§15.1)
Example dates `2025-03-04`, specimen accounts presented without qualification	Illustrative markers; dates inside 2026-09-15 → 2026-09-29

33. What Changed Because We Now Have the Real Dataset

The eight consequences that matter. Everything else in v2.1 is bookkeeping.

---

1 · `Transaction_ID` is not a primary key — provenance moves to the row

≈1,997,748 distinct IDs across 2,000,000 rows **[DATA-PRELIM]**. v2.0 would have quarantined the surplus; that instruction is withdrawn.

```
source_row_id  =  immutable CSV row ordinal  =  primary key & provenance anchor
Transaction_ID =  source-provided identifier =  indexed, returns a LIST
```

Every row is preserved (D1). Every evidence claim, graph edge, API response, export and document citation resolves to one exact source row — which is precisely what makes a freeze requisition defensible if two transfers share an ID. **Cost if retrofitted later: it touches the schema, the evidence object, the validator, the templates and the exports. Do it in Phase 1.**

> ⚠ The reported "2,252 duplicates" is the **row surplus**. Affected rows are between 2,253 and 4,504 depending on the repeat distribution. The profiler must emit the ID-repeat histogram and settle it.

---

2 · A 72-hour trace window returns the entire graph

With ≈80.4 outgoing transactions per account over 15 days, branching inside a 72 h window is ≈16:

```
hop 1 → 16      hop 2 → 259      hop 3 → 4,159      hop 4 → 66,885
                                                        ▲
                                 greater than the entire 24,873-account population
```

`TRACE_WINDOW_SEC` **72 h → 6 h** (b ≈ 1.34), with 1 h/6 h/24 h/72 h presets in the UI. The PS pattern completes in 3–15 minutes, so 6 hours is a 24× margin.

This also retroactively justifies the v2.0 design: on this data a plain BFS does not get slow, it returns everything — which is the same as returning nothing.

---

3 · The value-pruning floor was above the median transaction

Median transaction **₹899.35**, mean ₹1,760.88 **[DATA-PRELIM]**. v2.0 pruned branches below a flat **₹1,000** of traced value — more than half of all single transfers, and after proportional attribution most second- and third-hop values too. Genuine trails would have been truncated at hop 1–2 and the demo would have looked thin for reasons nobody could see.

```
TRACE_MIN_VALUE  =  max( ₹100 , 0.2% × initial_amount )
FREEZE_HOLDING_MIN  ₹10,000 → ₹2,000
SINK_MIN_AMOUNT     ₹50,000 → ₹10,000
```

Every rupee threshold in v2.0 was calibrated against an imagined amount distribution. All of them were re-checked against the observed one.

4 · The four "terminal signals" are probably one injected signal

```
foreign-IP rows                ≈ 2,654
automation-device rows         ≈ 2,654      (1,327 + 1,327)
crypto + wallet narration rows ≈ 2,654      (1,327 + 1,327)
```

Three independent counts landing on one number is not coincidence. The likely construction is ~2,654 transactions each carrying a foreign IP **and** an automation device **and** one of {crypto, wallet} narration **[ASSUME]**.

Consequence: v2.0's C5 blended four sub-signals as corroborating evidence. If they always co-occur, that quadruple-counts a single fact and lets one injected flag drive an account to CRITICAL. C5 becomes a **composite with a co-occurrence discount** driven by a profiler cross-tab, plus a hard `TERMINAL_SIGNAL_SOLO_CAP`. **This is the one open question we can answer ourselves, in Phase 1, without a mentor.**

---

5 · Terminal signals cannot find L1 or L2 — so behaviour must

The signals touch **0.13% of rows** and at most ~5,308 account-endpoints. Against the PS-described 1,500 mule accounts they plausibly mark only the L3 subset.

```
L1 collectors   → no terminal signal expected  → found by TOPOLOGY + VELOCITY
L2 distributors → no terminal signal expected  → found by FAN-OUT + SPLITTING
L3 terminals    → signal-bearing               → signals add PRECISION here
```

Where M3's hours go: fan-in/fan-out, matched pass-through, velocity, split consistency. **Where they do not go:** enriching the terminal keyword list. That list is already complete; it just cannot see two-thirds of the problem.

---

6 · "Collector" and "Distributor" are roles, not capabilities

The dataset makes this concrete: with ≈80 inbound and ≈80 outbound transactions per account on average, **almost every account both receives and sends**. A classifier keyed on "receives money" or "has no outgoing" would be noise.

```
Every account is profiled on BOTH sides, always.
Role = victim-relative position
     + dominant topology (degree_balance)
     + temporal flow behaviour
     + upstream/downstream relationships
     + terminal evidence
An account may be L1 in one case and L2 in another. Both are stored.
```

This propagates into the detection engine (§14), the classifier's ten inputs (§16), the MRI explanation, the node inspector, and the wording of the case diary.

---

7 · The graph is 5× bigger than estimated — and still small

≈99.8% of rows are their own unique (sender→receiver) pair, so edge aggregation buys almost nothing: **E ≈ 2.0 M, ~128 MB for both directions** rather than the ~52 MB v2.0 assumed. Comfortably affordable, but size the arrays correctly and do not spend time optimising an aggregation that cannot pay off.

The core architectural split is unchanged and now confirmed: **2,000,000 transaction rows in a columnar fact table, ≈24,873 accounts in an in-RAM integer graph.**

---

8 · We still have no labels — so every metric must say so

The PS describes 1,500 mule + 23,500 regular accounts. We observe ≈24,873 accounts, which is **consistent with** that population but does not prove it, and **we hold no labels**.

```
GROUND-TRUTH LABEL AVAILABILITY : UNKNOWN (mentor Q1, OPEN)
LABELS IN OUR POSSESSION        : NONE
EVALUATION REGIME               : HEURISTIC SELF-VALIDATION
```

Every published number carries its regime. Benchmark cells stay blank until measured. We do not describe a self-validated F1 as accuracy against ground truth, and we do not claim a signal proves fraud.

Stating this costs nothing and buys credibility — every judge knows a 36-hour prototype has limits; the teams that impress are the ones who already know which.

Phase-1 action list (before any tuning)

```
[ ] Checksum the CSV; record sha256 in the manifest
[ ] Run make profile  → data/artifacts/profile.json
[ ] Run make profile-check against §7.2; log every mismatch as P0
[ ] Resolve the Transaction_ID repeat histogram (surplus vs affected rows)
[ ] Produce the signal co-occurrence cross-tab → settles mentor Q15 ourselves
[ ] Read out-degree p50/p90/p99/max → set the hub threshold from data
[ ] Confirm unique_pairs → size the CSR arrays
[ ] Confirm the amount distribution → re-check every ₹ threshold in config.py
[ ] Replace the §7.0 status block with a verified stamp, or correct §7 and §33
```

> **One sentence for the team:** the architecture of v2.0 survived contact with the real data; its constants did not. Profile first, then tune — and never present an estimate as a measurement.

Document control

Version	Date	Author	Change
1.0	—	Team	Initial PRD
2.0	2026-10-01	Technical Lead	Build-ready: MRI formula, traversal algorithm, API contracts, Evidence Object + validators, S.94 BNSS framing, test matrix, RACI, hour-by-hour plan, risk register, demo script
2.1	**2026-10-01**	**Technical Lead**	**Dataset-grounded.** Rewritten §7 (Dataset Reality Check), §11 (provenance & duplicate policy), §14 (dual-sided features, matched pass-through), §16 (ten-input layer classifier), §25 (dataset-specific tests). Added FR-A05 profiling, FR-B02 dual-sided features, principles P7–P10, validators V11–V12, mentor questions Q13–Q17, §32 changelog, §33 dataset consequences. Recalibrated traversal window, value floors and hub guard against the observed distributions. All dataset figures marked **[DATA-PRELIM]** pending `make profile-check`.

> अभेद्य चक्र — the wheel that cannot be broken. Every rupee leaves a trace. Our job is to make it readable in two seconds — and to prove, row by row, that we did not make any of it up.